- 小白程式入門電子書
- 目錄
  - PART 1：認識程式
  - PART 2：核心概念
  - PART 3：進階之路
- 第1章：程式到底是什麼？
  - 一個平凡的早晨
  - 從蛋炒飯說起
  - 電腦：世界上最聽話的笨蛋
  - 生活中無所不在的程式思維
  - 關於那些迷思
    - 「我數學不好」
    - 「英文不好怎麼辦」
    - 「35歲會不會太晚」
    - 「需要天賦嗎」
    - 「需要很貴的電腦」
  - 錯誤訊息：最誠實的朋友
  - 程式改變了什麼
  - 第一章的最後
- 第2章：程式語言大觀園
  - 選擇的焦慮
  - 語言，只是工具
  - Python：像在寫詩的程式語言
  - JavaScript：網頁的魔法師
  - 其他語言的故事
    - Java：企業的可靠夥伴
    - Go：速度與簡潔的平衡
    - Swift：蘋果生態系的語言
  - 如何選擇你的第一個語言
  - 語言的偏見與真相
    - 「Python太慢了」
    - 「JavaScript不是真正的程式語言」
    - 「Java過時了」
    - 「C++太難了」
  - 超越語言的思考
  - 你的第一步
  - 寫在第二章的最後

- 第3章：像電腦一樣思考
  - 思維的轉換
  - 分解：大象要一口一口吃
  - 順序思維：第一件事永遠是第一件事
  - 條件思維：如果...就...否則...
  - 迴圈思維：重複，但要知道何時停止
  - 資料思維：一切都是資料
  - 抽象思維：見樹又見林
  - 除錯思維：錯誤是最好的老師
  - 模式識別：發現重複的力量
  - 寫在第三章的最後
- 第4章：資料與變數
  - 盒子的比喻
  - 資料的形態
    - 數字：精確的朋友
    - 文字：有故事的資料
    - 布林：是與否的世界
    - 清單：井然有序的集合
  - 命名的藝術
  - 資料的生命週期
    - 誕生
    - 成長與改變
    - 死亡
  - 型態的重要性
  - 資料的靈活運用
    - 組合不同型態
    - 資料轉換
    - 資料驗證
  - 資料告訴我們的故事
  - 寫在第四章的最後
- 第5章：自動化你的生活
  - 重複的詛咒
  - 第一個自動化：批次改名
  - Excel自動化：解放小美
  - 郵件自動化：別再複製貼上
  - 網路爬蟲：資訊自動收集
  - 檔案整理：數位斷捨離
  - 定時任務：讓電腦記得你會忘記的事

    - 自動化的哲學
    - 寫在第五章的最後
- 第6章：創造你的第一個專案
    - 從點到線，從線到面
    - 專案的誕生：從需求開始
    - 設計架構：畫出藍圖
    - 第一步：資料結構
    - 核心功能：新增帳目
    - 統計分析：讓數據說話
    - 預算管理：自我約束
    - 使用者介面：讓程式活起來
    - 除錯與改進：專案的成長
    - 部署與分享：讓別人使用你的作品
    - 專案的意義
    - 寫在第六章的最後
- 第7章：程式設計的職業道路
    - 咖啡廳裡的職涯討論
    - 工程師的光譜
    - 前端工程師：網頁的藝術家
    - 後端工程師：系統的建築師
    - 全端工程師：瑞士刀
    - 資料工程師：數據管道工
    - 轉職的真實路徑
        - 第0-3個月：打基礎
        - 第4-6個月：選定方向
        - 第7-9個月：作品集
        - 第10-12個月：求職準備
    - 第一份工作的真相
        - 1. 新創公司
        - 2. 接案/自由工作
        - 3. 大公司的約聘
        - 4. 內部轉職
    - 薪資的現實
    - 工程師的日常真相
    - 持續學習的宿命
    - 不當工程師也很好
    - 寫在第七章的最後
- 第8章：學習資源與社群

    - 資訊焦慮的時代
    - 免費資源的寶庫
        - YouTube：視覺學習者的天堂
        - GitHub：程式碼的圖書館
        - 線上練習平台
    - 付費課程的價值
        - Udemy：課程超市
        - Coursera：學院派
        - 台灣的選擇
    - 社群的力量
        - Facebook社團
        - Discord / Slack
        - 實體聚會
    - 部落格與寫作
        - 為什麼要寫
        - 寫什麼
        - 在哪寫
    - 播客與新媒體
        - 推薦播客
        - Twitter/X 追蹤
    - 學習資源的斷捨離
        - 資源評估矩陣
        - 個人學習系統
    - 寫在第八章的最後
- 第9章：學習程式的正確心態
    - 深夜的崩潰
    - bug是最好的老師
    - 冒牌者症候群
        - 如何面對冒牌者症候群
    - 學習高原期
        - 突破高原期的方法
    - 完美主義的陷阱
        - Done is better than perfect
    - 持續學習的智慧
        - 學習的優先順序
        - FOMO vs JOMO
    - 失敗是功能，不是bug
        - 重新定義失敗

    - 找到你的為什麼
    - 慶祝小勝利
    - 寫在第九章的最後
- 第10章：從零到一的行動計畫
    - 最後一堂課
    - 第一天：選擇你的武器
    - 第一週：建立習慣
    - 第一個月：打地基
    - 第一季：看見成長
    - 關鍵決策點
        - 路線A：深化現有技能
        - 路線B：橫向擴展
        - 路線C：準備轉職
    - 避開陷阱
    - 建立你的學習系統
        - 每日儀式
        - 週末專案日
        - 月度回顧
    - 找到你的節奏
    - 第一年後
    - 現在，立刻，馬上
    - 最後的話
    - 寫給一年後的你
    - 寫在全書的最後
- 感謝你讀到這裡
    - 這不是結束，而是開始
    - 加入桑尼的學習社群
        - 追蹤 Instagram
        - 加入 LINE 官方帳號
    - 期待你的第一步
        - 王子公主請上車，從這裡啟程

小白程式入門
電子書

從零開始
用故事帶你踏入程式世界

- 10 個章節完整學習路徑
- 真實故事引導入門
- 從觀念到實作一次搞懂

```
def learn():
    start()
    practice()
    grow()
```

桑尼資料科學
Data Sunnie

小白程式入門電子書

從零開始，用故事帶你踏入程式世界

作者：桑尼資料科學 | Data Sunnie

目錄

PART 1：認識程式

章節	標題	你將學到
01	程式到底是什麼？	打破迷思，理解程式的本質
02	程式語言大觀園	認識各種語言，找到適合你的
03	像電腦一樣思考	培養運算思維與邏輯能力

PART 2：核心概念

章節	標題	你將學到
04	資料與變數	程式如何儲存與處理資訊
05	自動化你的生活	用程式解決日常問題
06	創造你的第一個專案	動手實作，從想法到成品

PART 3：進階之路

章節	標題	你將學到
07	程式設計的職業道路	了解產業與職涯發展
08	學習資源與社群	找到持續學習的方法
09	學習程式的正確心態	克服挫折，建立成長思維

章節	標題	你將學到
10	從零到一的行動計畫	制定你的學習路線圖

> 給讀者的話
>
> 這本書不會教你成為程式大師，但會幫你跨出最重要的第一步。
>
> 每一章都是一個故事，每個故事都是一把鑰匙。
>
> 準備好了嗎？讓我們開始吧。

桑尼資料科學 | Data Sunnie

第1章：程式到底是什麼？

> 「寫程式很難嗎？」
>
> 這是我最常被問到的問題。每次聽到，我總會微笑著回答：「你知道嗎？你每天都在寫程式，只是用的不是電腦語言而已。」

一個平凡的早晨

讓我跟你分享一個故事。

小美，35歲，行銷經理。每天早上七點，鬧鐘準時響起。她習慣性地按下貪睡鈕，給自己額外的十分鐘。這個簡單的動作，其實就是一段完美的程式：檢查時間、判斷是否該起床、執行相應的動作。

那天早上，小美一邊刷牙一邊想著昨天同事展示的「魔法」——原本需要她花三個小時整理的報表，同事用幾行程式碼，三分鐘就完成了。

「如果我也會這個，生活會變成什麼樣子？」她看著鏡中的自己，牙刷停在半空中。

但隨即，一連串的疑慮湧上心頭：「我數學不好」、「英文也普通」、「35歲才開始會不會太晚」...

如果你也曾有過這些想法，請讓我告訴你一個秘密——這些都不是問題。真的。

從蛋炒飯說起

你會做蛋炒飯嗎？或者，你至少能想像怎麼做蛋炒飯吧？

準備材料、打蛋、熱油、炒蛋、加飯、調味、起鍋。每個步驟都有其順序，每個動作都有其目的。如果我告訴你，這就是程式設計的全部，你相信嗎？

程式，說穿了，就是告訴某個東西（可能是電腦、可能是機器、可能是另一個程式）該做什麼、怎麼做、什麼時候做。就像食譜告訴你如何做出一道菜，就像樂譜告訴演奏者如何演奏一首曲子。

當你教朋友做蛋炒飯時，你已經在「編程」了——只是你的執行者是人類，而不是電腦。

電腦：世界上最聽話的笨蛋

這個標題可能讓你意外。我們總認為電腦很聰明——它能在一秒內計算數百萬次、能記住海量資訊、能打敗世界棋王。但我要告訴你另一個角度的真相。

想像你有一個朋友，他：

- 完全不懂暗示
- 不會舉一反三
- 只認識「是」和「否」
- 需要你把每件事都說得清清楚楚

這就是電腦。

當你告訴人類「去買個漢堡」，他會自動理解需要帶錢、知道去哪裡買、懂得怎麼點餐。但對電腦來說？你需要告訴它什麼是錢、什麼是漢堡、什麼是「買」、如何找到店家、如何與人互動...每一個我們視為理所當然的細節。

有趣的是，正是這種「笨」，讓電腦變得可靠。它不會自作聰明，不會隨意解讀，給它同樣的指令，永遠會得到同樣的結果。這種可預測性，正是我們能夠信任電腦處理重要工作的原因。

就像小美發現的——當她終於理解電腦需要精確指令時，反而鬆了一口氣。「原來不是我不夠聰明，而是需要學會用電腦能懂的方式說話。」

生活中無所不在的程式思維

每天早上，你的鬧鐘在執行這樣的邏輯：

```
如果現在是設定時間：
    響鈴
    如果使用者按下貪睡：
        等待十分鐘
        重新開始
    否則：
        停止
```

每次使用Google Maps，背後都在進行複雜的運算：收集你的位置、分析所有可能路線、比較時間和距離、選出最佳方案、即時調整。這個我們習以為常的功能，其實是無數個「如果...就...」判斷的組合。

甚至連自動販賣機都在執行程式：等待投幣、計算金額、判斷是否足夠、執行相應動作。那個看似簡單的「投幣→選擇→取貨」流程，背後是嚴謹的邏輯判斷。

你看，程式思維從來都不陌生。它就在我們的日常生活中，只是我們從未用這個角度去看待它。

關於那些迷思

「我數學不好」

讓我問你：買東西算找零需要微積分嗎？做菜調味需要線性代數嗎？

大部分的程式設計，用到的數學不會超過小學程度。加減乘除、比大小、數數字——這就是90%程式工作需要的全部數學。

當然，如果你要開發3D遊戲引擎或研究人工智慧，那是另一回事。但對於想要自動化日常工作、建立網站、開發應用程式的人來說，會用計算機就夠了。

小美後來跟我說：「我發現寫程式的邏輯思考，反而讓我的數學變好了。因為我開始理解數字背後的意義，而不只是死記公式。」

「英文不好怎麼辦」

程式語言用到的英文，大概只有50個單字。而且都是最簡單的：if（如果）、for（對於）、while（當）、true（真）、false（假）。

這些單字，你現在就認識了，不是嗎？

更重要的是，理解這些單字背後的概念遠比拼寫它們重要。我認識許多優秀的工程師，他們的英文可能不流利，但這從未阻礙他們寫出優雅的程式。

「35歲會不會太晚」

若宮雅子，一位日本奶奶，81歲開始學程式，開發了一款專為銀髮族設計的手機遊戲。她說：「年齡給了我獨特的視角，我知道同齡人需要什麼。」

年齡從來不是限制，反而可能是優勢。你有豐富的人生經驗、明確的學習目標、更好的耐心。你知道自己為什麼要學，這比什麼都重要。

我見過太多30、40甚至50歲轉職成功的例子。他們的共同點不是年輕，而是清楚知道自己要什麼，並願意為此努力。

「需要天賦嗎」

如果寫程式需要天賦，那開車、做菜、學語言是不是也都需要天賦？

程式設計是一項技能，就像其他任何技能一樣。一開始會笨拙、會犯錯、會覺得困難。但隨著練習，你會越來越熟練，直到有一天，它變成你的第二天性。

我記得我的第一個程式，就是在螢幕上顯示「Hello World」。那兩個單字出現時，我興奮得像個孩子。現在回想起來，那個簡單到不能再簡單的程式，卻是一切的開始。

「需要很貴的電腦」

你現在用來閱讀這本書的設備，就足夠開始學習程式了。

真的，就是這麼簡單。

甚至，你連軟體都不用安裝。打開瀏覽器，進入任何一個線上程式環境，就可以開始寫你的第一行程式碼。我認識的許多工程師，他們的第一個程式就是在十年前的舊筆電上寫的。

錯誤訊息：最誠實的朋友

新手最害怕看到紅色的錯誤訊息。但讓我告訴你一個秘密——錯誤訊息是你最好的老師。

當電腦說「找不到檔案」，它在告訴你確切的問題所在。當它說「語法錯誤」，它在指出需要修正的地方。不像人類會委婉、會暗示，電腦的錯誤訊息永遠直接而誠實。

小美第一次看到錯誤訊息時嚇了一跳，以為自己做錯了什麼嚴重的事。後來她發現，那只是電腦在說：「我不懂你的意思，能說清楚一點嗎？」

學會閱讀錯誤訊息，就像學會聽懂電腦的語言。它不是在責備你，而是在幫助你。

程式改變了什麼

回到小美的故事。

三個月後，她寫了第一個自動化程式——自動整理每週的銷售報表。原本需要半天的工作，現在只要按個按鈕。她說：「不是程式有多厲害，而是我終於理解了，重複的工作就該交給電腦。」

六個月後，她開始幫同事解決各種自動化需求。不是因為她變成了工程師，而是因為她學會了用程式思維看待問題。

一年後，她獲得了升遷。老闆說：「不只是因為妳會寫程式，而是妳展現了解決問題的新思維。」

程式改變的，不只是工作效率，更是看待世界的方式。當你開始用程式思維思考，你會發現處處都有可以優化的流程、可以自動化的任務、可以簡化的複雜。

第一章的最後

如果你讀到這裡，你已經理解了程式的本質——它不是神秘的咒語，不是天才的專利，而是一種表達邏輯的方式，一種與機器溝通的語言。

你不需要成為全職工程師才值得學程式。就像你不需要成為作家才值得學寫字，不需要成為音樂家才值得學樂器。程式，可以只是你工具箱裡的另一個工具，幫助你更好地完成工作、解決問題、創造價值。

還記得開頭小美的疑問嗎？「如果我也會這個，生活會變成什麼樣子？」

現在，我想問你同樣的問題。不是「你能不能學會」——你當然能。而是「當你學會後，你想用它做什麼？」

這個答案，只有你知道。而這個答案，將成為你學習路上最強大的動力。

下一章，我們將一起探索不同程式語言的世界。不是要你全部學會，而是要找到最適合你的那一個。就像選擇一把趁手的工具，關鍵不在於工具本身，而在於你想創造什麼。

準備好了嗎？讓我們繼續這趟旅程。

每一個改變世界的程式，都始於某個人的第一行程式碼。 而你的第一行，可能就是改變你自己世界的開始。

第2章：程式語言大觀園

> 「我應該學哪個程式語言？」
>
> 如果說上一章的問題是最常被問的，那這個問題絕對是第二名。而我的答案總是：「你想用它來做什麼？」

選擇的焦慮

記得小美第一次打開程式學習網站時的表情。

Python、JavaScript、Java、C++、Ruby、Go、Swift...螢幕上密密麻麻的選項讓她眼花撩亂。每個語言旁邊都寫著聽不懂的特色：「物件導向」、「函數式」、「強型別」、「直譯式」...

「這就像站在一家有200種口味的冰淇淋店前，」她苦笑著說，「選擇太多，反而不知道該選什麼。」

她花了整整一週研究「最好的程式語言」，看了無數篇比較文章，每一篇都說得頭頭是道，但看完後她更迷茫了。有人說Python最簡單，有人說JavaScript最實用，有人說Java找工作最容易...

直到有一天，她問了我那個關鍵的問題：「我應該學哪個？」

而我反問她：「你想解決什麼問題？」

語言，只是工具

讓我分享一個可能會讓你驚訝的事實：對於大部分的程式任務，用哪個語言其實沒那麼重要。

這就像問：「我該學開手排車還是自排車？」答案是——重要的不是車子的類型，而是你要去哪裡。

每個程式語言，本質上都在做同樣的事：

- 儲存資料
- 做判斷
- 重複動作
- 與外界溝通

差別只在於表達方式和擅長的領域。就像中文適合寫詩，英文適合寫科學論文，但兩者都能表達「我愛你」。

讓我用一個簡單的例子說明。假設我們要讓電腦說「Hello, 小美」：

Python的方式：

```
name = "小美"
print(f"Hello, {name}")
```

JavaScript的方式：

```
let name = "小美";
console.log(`Hello, ${name}`);
```

Java的方式：

```
String name = "小美";
System.out.println("Hello, " + name);
```

看出來了嗎？概念完全一樣，只是「口音」不同。

Python：像在寫詩的程式語言

如果程式語言是樂器，Python就是鋼琴——優雅、直觀、適合初學者，但也能演奏出大師級的作品。

我永遠記得小美第一次寫Python時的反應：「等等，就這樣？不用分號？不用大括號？這真的是程式嗎？」

```
# 檢查今天是否該運動
體重 = 70
目標體重 = 65

if 體重 > 目標體重:
    print("該運動了！")
else:
    print("保持下去！")
```

Python的美在於它的簡潔。它去掉了所有不必要的符號，讓你專注在邏輯本身。這就像寫作時，你不需要先學習複雜的文法規則，而是直接開始表達想法。

Python特別適合：

- 數據分析（「這個月哪個產品賣最好？」）
- 自動化任務（「幫我整理這1000個檔案」）
- 人工智慧（「訓練一個認識貓的程式」）
- 網路爬蟲（「收集所有競爭對手的價格」）

小美最後選擇了Python，不是因為它「最好」，而是因為她的第一個目標是自動化Excel報表。三週後，她寫出了第一個實用程式——自動從20個分店的銷售檔案中提取數據，生成週報。原本要花她整個Monday早上的工作，現在只要按個按鈕。

JavaScript：網頁的魔法師

如果說Python是鋼琴，那JavaScript就是吉他——隨處可見、容易上手、而且超級酷炫。

「你現在看到的每個網頁，背後都有JavaScript在運作，」我告訴小美的同事阿威，一個想要創業的設計師。

阿威原本是平面設計師，他想要的不是處理數據，而是讓他的設計「活起來」。當他第一次用JavaScript讓按鈕被點擊時改變顏色，他的眼睛亮了起來。

```
// 當按鈕被點擊時
按鈕.addEventListener('click', function() {
    if (心情 === "開心") {
        背景.style.color = "黃色";
        播放音樂("快樂頌");
    } else {
        背景.style.color = "藍色";
        顯示訊息("需要一個擁抱嗎？");
    }
});
```

JavaScript的魔力在於「即時回饋」。你改一行程式碼，重新整理瀏覽器，立刻看到效果。這種即時的視覺回饋，對創意工作者來說特別有吸引力。

JavaScript特別適合：

- 網頁互動（「讓這個選單滑出來」）
- 前端開發（「做一個漂亮的購物網站」）
- 後端開發（「處理用戶登入」）——是的，JavaScript前後端通吃
- 手機App（React Native讓你用JavaScript寫App）

六個月後，阿威用JavaScript做出了他的第一個作品集網站。不是靜態的圖片展示，而是會動、會回應、會說故事的互動體驗。他說：「我終於能把腦中的創意完整呈現出來了。」

其他語言的故事

Java：企業的可靠夥伴

我有個朋友老陳，在銀行工作。當他決定學程式時，毫不猶豫地選了Java。

「銀行系統不能出錯，」他說，「Java就像穿西裝的管家，規規矩矩，但絕對可靠。」

Java要求你明確定義每個變數的類型，就像銀行要求每筆交易都要有明確的記錄。這看似麻煩，但當系統變大時，這種嚴謹能救你一命。

Go：速度與簡潔的平衡

小李是個系統工程師，他需要處理每秒數萬筆請求。他選擇了Go。

「Go就像瑞士軍刀，」他形容，「簡單、高效、專注於把事情做好。」

Go的並發處理能力驚人，就像一個超級協調者，能同時處理成千上萬件事而不會搞混。

Swift：蘋果生態系的語言

如果你的夢想是做iOS App，Swift是唯一的選擇。就像要在法國當廚師，你最好會說法語。

如何選擇你的第一個語言

經過這麼多故事，你可能還是在想：「所以我該選哪個？」

讓我給你一個簡單的指南：

如果你想要：

快速看到成果 → JavaScript 打開瀏覽器就能開始，改了就能看到效果。

處理數據、自動化工作 → Python 簡潔優雅，大量現成工具，適合解決實際問題。

找傳統大公司的工作 → Java 企業愛Java，工作機會多，薪水穩定。

做**iOS App** → Swift 別無選擇，但也是個優雅的語言。

打好基礎、理解底層 → C 困難但值得，像學武功要先蹲馬步。

但最重要的建議是：不要糾結太久。

記住，程式語言不是終身伴侶，你不需要「從一而終」。大部分工程師都會3-5種語言，而且學了第一個之後，學第二個會快很多。

就像小美，她從Python開始，一年後因為想做網頁，很自然地學了JavaScript。她說：「第二個語言只花了一個月就上手，因為概念都是相通的。」

語言的偏見與真相

在程式世界裡，有很多偏見和迷思。讓我來打破幾個：

「Python太慢了」

是的，Python執行速度比C慢。但你知道嗎？Instagram用Python處理數億用戶的照片。速度很重要，但開發速度更重要。用Python兩天寫完的程式，C可能要兩週。

「JavaScript不是真正的程式語言」

Netflix、PayPal、Uber的後端都大量使用JavaScript。如果它不是「真正」的程式語言，這些公司怎麼敢把幾億美金的生意交給它？

「Java過時了」

全世界有900萬個Java開發者，Android App大部分用Java寫。過時？我看是穩如泰山。

「C++太難了」

遊戲產業愛死C++了。你玩的3A大作，底層引擎幾乎都是C++。難？是的。但值得？看你想做什麼。

超越語言的思考

讓我告訴你一個秘密：真正的程式高手，不是精通某個語言的人，而是能夠超越語言思考的人。

當你理解了「變數」的概念，你就能在任何語言中使用它。 當你學會了「迴圈」的邏輯，你就能用任何語法表達它。 當你掌握了「問題分解」的能力，你就能用任何工具解決它。

這就像一個真正的廚師，給他任何廚房、任何食材，他都能做出美味的料理。工具只是工具，創造力才是關鍵。

小美現在已經能用Python和JavaScript工作了。但更重要的是，她學會了「程式思維」。她看問題的角度變了，解決問題的方法變了，甚至思考的方式都變了。

「語言只是表達方式，」她說，「重要的是你想表達什麼。」

你的第一步

如果你讀到這裡還在猶豫，讓我給你一個最簡單的建議：

閉上眼睛，想像一個你想解決的問題。

是想要自動化無聊的工作？選Python。 是想要做個人網站？選JavaScript。 是想要開發App？看你用iPhone還是Android。

然後，今天就開始。

不要再研究了，不要再比較了，不要再等「最佳時機」了。

打開電腦，選一個語言，寫下你的第一行程式碼。哪怕只是讓電腦說「Hello」，那也是你改變的開始。

記住阿威的話：「我糾結了三個月該學什麼，但真正改變我的，是我終於開始寫的那一天。」

寫在第二章的最後

程式語言就像世界各地的語言，每種都有its own beauty，都有它的文化和哲學。

選擇哪一種開始，真的沒那麼重要。重要的是開始。

下一章，我們要談談更基礎的東西——如何像電腦一樣思考。因為無論你選擇什麼語言，邏輯思維都是相同的。

但在那之前，我想留給你一個思考：

如果程式語言是工具，那你想用這個工具創造什麼？

這個答案，比選擇哪個語言重要一萬倍。

「*The best programming language is the one you actually use.*」 最好的程式語言，是你真正在用的那一個。

第3章：像電腦一樣思考

> 「我的大腦就是轉不過來。」
>
> 小美盯著螢幕上的程式碼，挫折地說。她已經學會了Python的基本語法，但當要解決一個實際問題時，她不知道從何下手。

思維的轉換

那是一個週五下午，小美想寫一個程式來整理客戶名單。需求很簡單：把重複的刪除，按照消費金額排序，然後標記VIP客戶。

她知道Python有排序功能，也知道怎麼刪除重複，但就是不知道怎麼把這些組合起來。

「我知道要做什麼，」她說，「但不知道怎麼告訴電腦。」

這就是問題所在。學會語法就像學會了字母，但要寫出文章，你需要學會思考的方式。

人類的思考是跳躍的、直覺的、模糊的。 電腦的思考是線性的、邏輯的、精確的。

要寫程式，你需要學會在這兩種思維模式之間切換。

分解：大象要一口一口吃

還記得那個古老的問題嗎？「如何吃掉一頭大象？」答案是：「一口一口吃。」

程式設計的第一個思維模式就是：分解。

看看小美的需求：「整理客戶名單」。這對人類來說是一個任務，但對電腦來說太模糊了。

我引導她把任務分解：

```
整理客戶名單
├── 讀取客戶資料
├── 處理資料
│   ├── 刪除重複
│   ├── 計算消費總額
│   └── 排序
├── 標記VIP
│   └── 如果消費 > 10000，標記為VIP
└── 輸出結果
```

「哦！」小美恍然大悟，「原來要這樣一步一步來。」

這就像做菜。你不會說「做一頓飯」，而是：

1. 決定菜單
2. 準備食材
3. 處理食材（洗、切、醃）
4. 烹煮
5. 擺盤

每個步驟都是具體的、可執行的。

順序思維：第一件事永遠是第一件事

電腦很笨，它不會「同時」做很多事（看起來同時，其實是快速切換）。它只會一步一步，從上到下執行。

小美一開始寫的程式是這樣的：

```
排序客戶名單()
刪除重複()
讀取檔案()
標記VIP()
```

「為什麼不能運作？」她困惑地問。

問題很明顯——她在讀取檔案之前就想要排序。這就像還沒買菜就想炒菜。

人類可以在腦中跳來跳去：「我要排序...啊對了要先讀檔...然後刪除重複...」但電腦不行。

正確的順序應該是：

```
# 1. 先有資料
客戶資料 = 讀取檔案()

# 2. 再處理資料
客戶資料 = 刪除重複(客戶資料)

# 3. 然後排序
客戶資料 = 排序(客戶資料)

# 4. 最後標記
標記VIP(客戶資料)
```

這種「順序思維」一開始很不自然，但慢慢地，你會發現它其實讓事情變簡單了。因為你被迫把複雜的任務整理成清晰的步驟。

條件思維：如果...就...否則...

生活充滿了選擇。程式也是。

但電腦的選擇必須是明確的、沒有模糊地帶的。

人類的思考：「如果客戶買得夠多，就給他VIP。」 電腦需要：「如果客戶消費金額大於10000元，就標記為VIP，否則標記為一般會員。」

看出差別了嗎？

- 「買得夠多」→「大於10000元」（明確的數字）

- 暗示的「不是VIP」→明確的「否則標記為一般會員」

小美在處理促銷邏輯時遇到了困難：

「滿千送百，但如果是VIP再打9折，如果又是生日月份再送生日禮...」她的頭都暈了。

我教她畫決策樹：

```
消費金額
├── 大於1000？
│   ├── 是 → 送100元折價券
│   │   └── 是VIP？
│   │       ├── 是 → 再打9折
│   │       └── 否 → 維持原價
│   └── 否 → 無優惠
└── 是生日月份？
    ├── 是 → 送生日禮
    └── 否 → 無生日禮
```

「原來可以這樣整理！」她興奮地說。把模糊的規則變成清晰的決策樹，程式碼就自然而然地寫出來了。

迴圈思維：重複，但要知道何時停止

如果要你數到100，你不會說：「1、2、3、4、5...」一直說到100。你會說：「從1數到100」。

這就是迴圈思維。

但電腦的迴圈需要三個要素：

1. 起點（從哪裡開始）
2. 終點（什麼時候停）
3. 步進（每次變化多少）

小美要處理1000個客戶資料，一開始她想：

```
處理第1個客戶
處理第2個客戶
處理第3個客戶
# ...天啊要寫1000行？
```

當她學會迴圈思維後：

```
for 客戶 in 所有客戶:
    處理(客戶)
```

兩行搞定。

但迴圈最難的是知道何時停止。無限迴圈是新手最常犯的錯誤。就像小美第一次寫的：

```
while 還有客戶:
    處理下一個客戶
    # 忘記更新"還有客戶"的狀態
```

程式永遠不會停，因為它永遠認為「還有客戶」。

這就像在跑步機上跑步，卻忘記設定要跑多久。你會一直跑到累垮。

資料思維：一切都是資料

這個思維轉變最抽象，但也最重要。

在電腦的世界裡，一切都是資料：

- 你的名字是資料
- 你的年齡是資料
- 你的照片是資料
- 甚至程式本身也是資料

小美一開始很困惑：「照片怎麼會是資料？」

我打開一張照片的原始碼給她看——密密麻麻的數字。每個數字代表一個像素的顏色。對電腦來說，蒙娜麗莎的微笑不過是幾百萬個數字的組合。

這個認知很重要，因為一旦你理解「一切都是資料」，你就能理解為什麼電腦可以：

- 壓縮檔案（改變資料的排列方式）
- 加密訊息（把資料變成另一種形式）
- 傳送影片（把資料切成小塊傳送）

更重要的是，你會開始用「資料」的角度看問題。

客戶不滿意？那是一個資料點。 銷售下滑？查看資料趨勢。 要不要推出新產品？讓資料說話。

抽象思維：見樹又見林

程式設計的最高境界是抽象思維——能夠在細節和全局之間自如切換。

還記得小美的客戶整理程式嗎？一開始她寫的是：

```
# 處理A公司的客戶
讀取A公司檔案()
刪除重複()
排序()
標記VIP()
輸出()

# 處理B公司的客戶
讀取B公司檔案()
刪除重複()
排序()
標記VIP()
輸出()

# 處理C公司的客戶...
```

她發現每個公司的處理流程都一樣，只是檔案名稱不同。這時候就需要抽象思維：

```
def 處理客戶資料(公司名稱):
    讀取檔案(公司名稱)
    刪除重複()
    排序()
    標記VIP()
    輸出()

# 現在處理任何公司都只要一行
處理客戶資料("A公司")
處理客戶資料("B公司")
處理客戶資料("C公司")
```

抽象就是找出共同模式，把變化的部分參數化。

這就像食譜。你不會為每種蛋糕寫一個完全不同的食譜，而是有一個基本蛋糕食譜，然後說：「如果要巧克力口味，加可可粉；如果要檸檬口味，加檸檬汁。」

除錯思維：錯誤是最好的老師

小美最怕看到錯誤訊息。紅色的文字讓她覺得自己做錯了什麼嚴重的事。

直到有一天，她的程式出現了這個錯誤：

```
IndexError: list index out of range
```

她panic了。但我要她深呼吸，然後一起分析：

「Index是索引的意思，out of range是超出範圍。所以電腦在告訴你：你要找的東西超出了清單的範圍。」

就像你有一個10格的置物櫃，卻要打開第11格。

她檢查程式碼，果然發現她的客戶清單只有100個，但她卻想要處理第101個。

從那天起，她對錯誤訊息的態度變了。她開始把它們當作線索，而不是指責。

錯誤訊息就像醫生的診斷：

- 它告訴你哪裡出問題
- 它縮小了搜尋範圍
- 它給你改正的方向

學會「除錯思維」的人，會主動製造錯誤來測試程式的邊界。就像疫苗一樣，小劑量的錯誤能讓程式更強壯。

模式識別：發現重複的力量

人類大腦擅長模式識別，但在寫程式時，我們常常忘記運用這個能力。

小美有一天要處理公司的班表。她發現一個規律：

- 週一到週五是一個模式
- 週末是另一個模式
- 國定假日又是一個模式

與其為365天寫365個規則，她學會了識別和利用模式：

```
if 是平日():
    套用平日班表()
elif 是週末():
    套用週末班表()
elif 是國定假日():
    套用假日班表()
```

這種思維不只在程式設計有用。當你開始注意模式，你會發現：

- 客訴總是集中在某幾個問題
- 銷售高峰總是出現在特定時段
- 成功的專案總是有某些共同特徵

程式思維訓練你成為模式偵探。

寫在第三章的最後

「像電腦一樣思考」聽起來很冰冷，但其實是在學習一種新的表達方式。

就像學習繪畫改變你看世界的方式——你開始注意光影、比例、色彩。學習程式思維也會改變你看問題的方式——你開始注意順序、條件、重複、模式。

小美現在不只會寫程式了。她的工作方式完全改變了：

- 開會時，她會把討論內容整理成決策樹
- 做專案時，她會先分解成小任務
- 遇到問題時，她會系統性地除錯

她說：「我沒有變成電腦，我只是學會了另一種思考方式。需要的時候可以切換，就像會說兩種語言的人。」

這就是程式思維的真正價值。不是要你變成機器，而是要你掌握機器的語言，讓機器為你工作。

下一章，我們要談談程式的基礎材料——資料。就像廚師需要了解食材，程式設計師需要了解資料。

但在那之前，試著用今天學到的思維方式，重新看看你的日常工作。你會發現，很多事情都可以被分解、被排序、被自動化。

而這，正是改變的開始。

「學習程式，不是學習成為電腦， 而是學習如何與電腦合作。」

第4章：資料與變數

> 「變數到底是什麼？為什麼要叫變數？它會變嗎？」
>
> 阿威第一次聽到「變數」這個詞時，滿臉疑惑。作為一個設計師，他習慣視覺化的東西，但變數...看不見也摸不著。

盒子的比喻

我給阿威看了我桌上的收納盒。

「你看，這個盒子上貼著標籤『筆』，裡面放著各種筆。這個貼著『名片』，裡面放著收集來的名片。」

他點點頭。

「變數就是這樣的盒子。標籤是變數的名字，裡面的東西是變數的值。」

```
筆 = "原子筆"
名片 = "王小明的名片"
```

「但為什麼叫『變』數？」他追問。

我把筆盒裡的原子筆換成鉛筆。

「盒子還是同一個盒子，標籤還是『筆』，但內容變了。這就是為什麼叫變數——內容可以變。」

```
筆 = "原子筆"
# 過了一會兒...
筆 = "鉛筆"
# 盒子（變數）沒變，但內容（值）變了
```

阿威恍然大悟：「所以變數就像Photoshop的圖層名稱，名稱不變，但圖層內容可以隨時更換！」

沒錯，每個領域都有類似的概念，只是叫法不同。

資料的形態

如果變數是盒子，那資料就是放進盒子的東西。而不同的東西，需要不同的存放方式。

數字：精確的朋友

數字是最單純的資料。

```
年齡 = 35
薪水 = 50000
溫度 = 23.5
```

小美發現了一個有趣的事：「為什麼有時候數字有小數點，有時候沒有？」

這就像問：為什麼有時候我們說「3個人」而不是「3.5個人」？

- 整數（Integer）：完整的數字，用來數東西
- 浮點數（Float）：有小數點的數字，用來測量

你不會說你有2.5個小孩，但你會說今天氣溫25.5度。

程式也是這樣。庫存數量用整數，價格用浮點數。選對了，程式更清晰；選錯了，可能出現奇怪的結果。

文字：有故事的資料

```
姓名 = "小美"
公司 = "快樂行銷有限公司"
座右銘 = "每天進步1%"
```

文字（String）是用引號包起來的任何東西。

阿威問：「為什麼要用引號？」

「因為要告訴電腦：這是文字，不是指令。」

看看這個差別：

```
print(Hello)     # 電腦會找一個叫Hello的變數
print("Hello")   # 電腦知道要顯示Hello這個字
```

就像在對話中，我們用引號標示別人說的話： 小美說：「我要學程式。」

引號告訴讀者：這是原話，不是我的詮釋。

布林：是與否的世界

這是最簡單卻最強大的資料型態。

```
已婚 = True
有駕照 = False
是VIP = True
```

布林（Boolean）只有兩個值：True（真）或False（假）。

聽起來很限制，但想想看，多少重要決定都是二選一：

- 要不要買？
- 符不符合條件？
- 該不該執行？

小美在處理促銷活動時，大量使用布林：

```
滿千送百 = (消費金額 >= 1000)
可以打折 = 是會員 and 非特價品
需要補貨 = (庫存 < 10)
```

每個複雜的商業邏輯，拆解到最後都是一連串的是與否。

清單：井然有序的集合

```
員工名單 = ["小美", "阿威", "老陳", "小李"]
本月業績 = [120000, 98000, 150000, 87000]
待辦事項 = ["開會", "寫報告", "回覆客戶"]
```

清單（List）就像購物清單，可以放很多東西，而且有順序。

阿威特別喜歡清單，因為它像他的設計專案：

```
設計流程 = [
    "了解需求",
    "研究競品",
    "草圖發想",
    "設計初稿",
    "客戶反饋",
    "修改定稿"
]
```

清單的神奇之處在於，你可以：

- 加入新項目（append）
- 移除項目（remove）
- 重新排序（sort）
- 取出特定位置的項目（用索引）

就像整理你的書架，隨時可以加書、抽書、重新排列。

命名的藝術

「我可以把變數命名為a、b、c嗎？」小美問。

「可以，但不應該。」

看看這兩段程式：

```
# 糟糕的命名
a = 50000
b = 0.1
c = a * b

# 清晰的命名
月薪 = 50000
稅率 = 0.1
應繳稅額 = 月薪 * 稅率
```

哪個更容易理解？

好的變數名稱就像好的標籤，讓人一看就懂裡面裝什麼。

我給他們一個原則：「如果三個月後的你看不懂，那就是壞名字。」

阿威深有感觸：「就像我的設計檔案，如果命名為『最終版』、『最終版2』、『最最終版』，過幾天連我自己都搞不清楚哪個是真的最終版。」

資料的生命週期

變數不是永恆的。它們有生有死。

誕生

```
# 變數誕生的瞬間
客戶名稱 = "王大明"
```

成長與改變

```
訂單金額 = 1000
訂單金額 = 訂單金額 + 500  # 追加訂購
訂單金額 = 訂單金額 * 0.9   # 打九折
```

死亡

當程式結束，或變數超出範圍，它就消失了。

```
def 處理訂單():
    臨時折扣 = 0.8  # 這個變數只在函數內存在
    return 原價 * 臨時折扣

# 函數結束，臨時折扣消失了
print(臨時折扣)  # 錯誤！找不到這個變數
```

小美說：「這就像會議室的白板，會議中寫滿東西，會議結束就擦掉了。」

正是如此。理解變數的生命週期，能幫你避免很多錯誤。

型態的重要性

有一天，小美遇到了一個詭異的問題：

```
數量 = input("請輸入購買數量：")  # 使用者輸入10
價格 = 100
總價 = 數量 * 價格
```

她期待總價是1000，結果卻出現錯誤。

問題在哪？`input()`回傳的是文字，不是數字。「10」（文字）和10（數字）對電腦來說是不同的東西。

```
"10" * 100  # 錯誤！不能用文字乘以數字
10 * 100    # 正確！得到1000
```

解決方法：

```
數量 = int(input("請輸入購買數量："))  # 轉換成整數
```

這個錯誤教會了小美一個重要觀念：資料型態很重要。

就像cooking，鹽和糖看起來都是白色粉末，但你絕不會想搞混它們。

資料的靈活運用

當你理解了資料的本質，你就能靈活運用它們。

組合不同型態

```
# 組合成有意義的資訊
姓名 = "小美"
年齡 = 35
部門 = "行銷部"

員工資訊 = f"{姓名}，{年齡}歲，{部門}"
# 結果："小美，35歲，行銷部"
```

資料轉換

```
# 數字轉文字
金額 = 1000
顯示文字 = "總計：" + str(金額) + "元"

# 文字轉數字
使用者輸入 = "25"
實際年齡 = int(使用者輸入)

# 任何東西轉布林
有庫存 = bool(庫存數量)  # 0會變成False，其他數字變成True
```

資料驗證

```
def 檢查email(email):
    有小老鼠 = "@" in email
    有點 = "." in email
    return 有小老鼠 and 有點
```

```
客戶email = "wang@example.com"
if 檢查email(客戶email):
    發送郵件()
else:
    顯示錯誤("Email格式不正確")
```

資料告訴我們的故事

老陳在銀行工作，他對資料有不同的看法：

「每筆資料背後都是一個故事。交易金額不只是數字，它可能是某人的第一份薪水、結婚基金、或是孩子的學費。」

這個觀點改變了大家看待資料的方式。

```
# 不只是資料，而是故事
交易紀錄 = {
    "金額": 30000,
    "時間": "2024-06-01",
    "備註": "女兒大學註冊費",
    "帳戶餘額": 150000
}

# 資料可以回答問題
平均消費 = sum(月消費) / len(月消費)
最大支出 = max(所有支出)
存款趨勢 = 這個月餘額 - 上個月餘額
```

資料不是冰冷的數字和文字，而是幫助我們理解世界、做出決策的工具。

寫在第四章的最後

變數和資料，聽起來很技術，但其實是我們每天都在做的事：

- 記住名字（字串變數）
- 計算花費（數字變數）
- 做決定（布林變數）
- 列清單（清單變數）

阿威現在設計網頁時，會自然地思考：「這裡需要什麼資料？使用者名稱是字串，按讚數是整數，是否已登入是布林...」

小美處理報表時，不再把數據當成無意義的數字，而是能看出背後的模式和趨勢。

程式設計的核心，就是操作資料。而理解資料的本質，就像廚師理解食材——你知道每種食材的特性，就能創造出美味的料理。

下一章，我們要看看如何用這些資料，加上前面學的邏輯思維，來自動化你的日常工作。

因為當你能夠掌控資料，你就能掌控資訊。 而在這個資訊時代，掌控資訊就是掌控未來。

準備好了嗎？讓我們看看程式如何改變你的工作方式。

「*Data is the new oil.*」 資料是新時代的石油。 但只有懂得提煉的人，才能釋放它的價值。

第5章： 自動化你的生活

> 「我每個星期一要花三個小時整理報表，每個月底要花一整天對帳，每天要手動備份檔案...」
>
> 小美算了算，她每個月有將近40個小時在做重複的工作。「如果這些時間可以省下來...」

重複的詛咒

讓我問你一個問題：你一天中有多少時間在做「昨天也做過」的事？

不是創意發想，不是解決新問題，而是機械性的重複：

- 複製貼上
- 改檔名
- 整理資料
- 發制式郵件
- 填表格

如果你仔細計算，答案可能會讓你驚訝。

小美第一次意識到這個問題，是在一個週一早上。她打開電腦，開始她的「週一儀式」：

1. 從20個分店下載銷售報表
2. 打開每個Excel檔案
3. 複製特定欄位
4. 貼到總表
5. 計算總和
6. 製作圖表
7. 匯出PDF
8. 發郵件給主管

三個小時後，她完成了。就像上週一，上上週一，以及過去兩年的每個週一。

「一定有更好的方法，」她心想。

有的。它叫自動化。

第一個自動化：批次改名

阿威遇到了一個問題。客戶傳來200張產品照片，檔名亂七八糟：

- IMG_001.jpg
- 照片1.png
- 未命名-2.jpeg
- Copy of DSC1234.JPG

他需要統一改成：

- 產品_001.jpg
- 產品_002.jpg
- 產品_003.jpg
- ...

手動改？要改到什麼時候？

我教他寫了第一個自動化程式：

```python
import os

資料夾路徑 = "產品照片"
檔案列表 = os.listdir(資料夾路徑)
計數器 = 1

for 舊檔名 in 檔案列表:
    if 舊檔名.endswith(('.jpg', '.jpeg', '.png')):
        新檔名 = f"產品_{計數器:03d}.jpg"
        os.rename(舊檔名, 新檔名)
        計數器 += 1
        print(f"已改名：{舊檔名} → {新檔名}")
```

五秒鐘，200個檔案改好了。

阿威盯著螢幕，久久說不出話。然後他說了一句我永遠不會忘記的話：

「這三行程式碼，抵得上我兩個小時的工作。」

不，是抵得上他未來所有需要改檔名的時間。因為程式寫一次，可以用一輩子。

Excel自動化：解放小美

回到小美的週一地獄。

我們一起分析了她的工作流程，發現全部都可以自動化：

```python
import pandas as pd
import glob

# 找出所有分店的報表
所有報表 = glob.glob("分店報表/*.xlsx")

# 創建一個空的總表
總表 = pd.DataFrame()

# 讀取每個報表，合併到總表
for 報表路徑 in 所有報表:
    分店名稱 = 報表路徑.split("/")[-1].replace(".xlsx", "")

    # 讀取Excel
    資料 = pd.read_excel(報表路徑)

    # 加上分店名稱
    資料["分店"] = 分店名稱
```

```
    # 合併到總表
    總表 = pd.concat([總表, 資料])

# 計算各種統計
總銷售額 = 總表["銷售額"].sum()
平均客單價 = 總表["銷售額"].mean()
最佳產品 = 總表.groupby("產品")["數量"].sum().idxmax()

# 輸出結果
總表.to_excel("週報總表.xlsx", index=False)
print(f"本週總銷售額：{總銷售額:,.0f}")
print(f"平均客單價：{平均客單價:.2f}")
print(f"最暢銷產品：{最佳產品}")
```

第一次執行成功時，小美的反應是：「等等...就這樣？三小時的工作，10秒就完成了？」

是的，就這樣。

而且不只是快，還更準確。沒有複製貼上的錯誤，沒有公式拉錯，沒有漏掉的分店。

郵件自動化：別再複製貼上

老陳每天要發送帳單提醒給客戶。內容大同小異，只是名字和金額不同。

之前他是這樣做的：

1. 打開郵件範本
2. 複製內容
3. 改名字
4. 改金額
5. 改日期
6. 發送
7. 重複50次

現在：

```
import smtplib
from email.mime.text import MIMEText
from datetime import datetime

def 發送帳單提醒(客戶名稱, email, 金額, 到期日):
    主旨 = f"帳單到期提醒 - {客戶名稱}"
```

```
        內容 = f"""
        親愛的{客戶名稱}，

        提醒您，您本月的帳單金額為 ${金額:,}

        繳費期限：{到期日}

        請記得準時繳納，謝謝！

        祝好
        客服部
        """

        # 發送郵件
        發送郵件(email, 主旨, 內容)
        print(f"已發送提醒給 {客戶名稱}")

    # 讀取客戶清單
    客戶清單 = pd.read_excel("待繳費客戶.xlsx")

    for _, 客戶 in 客戶清單.iterrows():
        發送帳單提醒(
            客戶["姓名"],
            客戶["Email"],
            客戶["金額"],
            客戶["到期日"]
        )
```

50封郵件，一分鐘搞定。

但更重要的是心理上的解放。老陳說：「以前每天下午三點，我就開始焦慮要發郵件。現在我只要按個按鈕。」

網路爬蟲：資訊自動收集

小李的工作是市場研究，需要每天查看競爭對手的價格。

以前的流程：

1. 打開10個競品網站
2. 搜尋產品
3. 記錄價格
4. 輸入Excel
5. 每天重複

這不只無聊，還容易出錯。看錯一個數字，整個分析就歪了。

自動化解決方案：

```python
import requests
from bs4 import BeautifulSoup
import schedule

def 檢查競品價格():
    競品網址 = [
        "https://competitor1.com/product",
        "https://competitor2.com/product",
        # ...
    ]

    今日價格 = {}

    for 網址 in 競品網址:
        回應 = requests.get(網址)
        網頁 = BeautifulSoup(回應.text, 'html.parser')

        # 找出價格（每個網站結構不同）
        價格元素 = 網頁.find('span', class_='price')
        價格 = float(價格元素.text.replace('$', ''))

        商家名稱 = 網址.split('/')[2]
        今日價格[商家名稱] = 價格

    # 儲存到資料庫或Excel
    儲存價格(今日價格)

    # 如果有異常變動，發出警報
    for 商家, 價格 in 今日價格.items():
        if 價格變動超過10%:
            發送警報(f"{商家}價格異常變動！")

# 每天早上9點自動執行
schedule.every().day.at("09:00").do(檢查競品價格)
```

現在小李每天早上打開電腦，價格報告已經在信箱裡了。如果有重要變動，手機會立刻收到通知。

「我從資料收集員變成了資料分析師，」他說，「這才是我真正的價值所在。」

檔案整理：數位斷捨離

阿威的電腦桌面是一場災難。幾百個檔案散落各處，要找東西全靠搜尋。

我們寫了一個自動整理程式：

```python
import os
import shutil
from datetime import datetime

def 整理桌面():
    桌面路徑 = os.path.expanduser("~/Desktop")

    # 建立分類資料夾
    分類 = {
        "圖片": ['.jpg', '.png', '.gif', '.psd'],
        "文件": ['.doc', '.docx', '.pdf', '.txt'],
        "試算表": ['.xls', '.xlsx', '.csv'],
        "影片": ['.mp4', '.mov', '.avi'],
        "其他": []
    }

    # 建立資料夾
    for 資料夾名稱 in 分類.keys():
        資料夾路徑 = os.path.join(桌面路徑, 資料夾名稱)
        if not os.path.exists(資料夾路徑):
            os.makedirs(資料夾路徑)

    # 整理檔案
    for 檔案名稱 in os.listdir(桌面路徑):
        if os.path.isfile(os.path.join(桌面路徑, 檔案名稱)):
            檔案分類 = "其他"

            for 類別, 副檔名列表 in 分類.items():
                if any(檔案名稱.endswith(ext) for ext in 副檔名列表):
                    檔案分類 = 類別
                    break

            # 移動檔案
            來源 = os.path.join(桌面路徑, 檔案名稱)
            目標 = os.path.join(桌面路徑, 檔案分類, 檔案名稱)
            shutil.move(來源, 目標)
            print(f"已移動：{檔案名稱} → {檔案分類}/")
```

執行後，混亂的桌面瞬間井然有序。圖片在圖片資料夾，文件在文件資料夾。

「這就像請了一個數位管家，」阿威說。

定時任務：讓電腦記得你會忘記的事

人會忘記，電腦不會。

小美常常忘記備份重要檔案，直到有一天硬碟壞了，她才後悔莫及。

現在，她有了自動備份：

```
import shutil
import schedule
from datetime import datetime

def 自動備份():
    重要資料夾 = [
        "工作文件",
        "客戶資料",
        "財務報表"
    ]

    今天日期 = datetime.now().strftime("%Y%m%d")
    備份資料夾 = f"備份_{今天日期}"

    for 資料夾 in 重要資料夾:
        來源 = f"C:/Users/小美/{資料夾}"
        目標 = f"D:/備份/{備份資料夾}/{資料夾}"

        shutil.copytree(來源, 目標)
        print(f"已備份：{資料夾}")

    # 刪除30天前的備份（節省空間）
    清理老舊備份()

# 每天下班前自動備份
schedule.every().day.at("17:30").do(自動備份)

# 每週五額外備份到雲端
schedule.every().friday.at("18:00").do(備份到雲端)
```

現在她再也不用擔心資料遺失了。

自動化的哲學

自動化不只是技術，更是一種思維方式。

當你開始用自動化的眼光看世界，你會發現處處都有機會：

- 每天檢查的網站 → RSS訂閱或爬蟲
- 定期發送的報告 → 排程任務
- 重複的滑鼠點擊 → 腳本程式
- 例行的資料處理 → 批次處理

但自動化的真正價值，不在於省下的時間，而在於釋放的心智。

當你不需要記住「每週一要整理報表」，你的大腦就有空間思考「如何提升銷售」。

當你不需要手動備份檔案，你就不會焦慮資料安全。

當重複的工作都自動化了，你終於可以專注在真正重要的事：創造、思考、連結。

寫在第五章的最後

小美現在的工作日是這樣的：

早上打開電腦，昨天的銷售報告已經整理好了。 需要發送的郵件已經發出去了。 重要檔案已經備份了。 競品價格已經更新了。

她有整個上午的時間思考策略、創意發想、與團隊討論。

「我以前是資料的奴隸，」她說，「現在資料為我工作。」

這就是自動化的力量。它不會取代你，但會解放你。讓你從機器人變回人類。

記住：任何你做第二次的事，都值得考慮自動化。

因為生命太短暫，不該浪費在複製貼上。

下一章，我們要把這些片段的自動化，組合成一個完整的專案。你將創造你的第一個「作品」。

那將是你程式之旅的里程碑。

準備好了嗎？

「The first rule of any technology used in a business is that automation applied to an efficient operation will magnify the efficiency.」 - Bill Gates

自動化不是懶惰，而是智慧。 把重複交給機器，把創造留給自己。

第6章：創造你的第一個專案

> 「我學了變數、迴圈、條件判斷...但我還是不知道怎麼做出一個真正的東西。」
>
> 阿威看著螢幕上零散的程式碼片段，就像看著一堆樂高積木，不知道如何組合成城堡。

從點到線，從線到面

學程式就像學語言。

你學了單字（變數），學了文法（語法），甚至能造句（寫函數）。但要寫出一篇文章，那是另一個層次的挑戰。

專案，就是你的第一篇文章。

今天，我們要一起創造一個真正的專案——一個個人記帳程式。

為什麼選記帳？因為每個人都需要，而且它包含了程式設計的所有核心概念。

更重要的是，當你完成它，你會真的想用它。

專案的誕生：從需求開始

小美、阿威、老陳和小李坐在咖啡廳，討論他們想要什麼樣的記帳程式。

「市面上的記帳App太複雜了，」小美說，「我只想記錄花了多少，不需要那麼多功能。」

「我希望能看到每個月的趨勢，」老陳補充，「知道錢都花到哪裡去了。」

「最好能設定預算提醒，」阿威說，「快超支的時候警告我。」

「還要能匯出報表，」小李說，「年底報稅用得到。」

這就是需求分析——專案的第一步。不是寫程式，而是搞清楚要解決什麼問題。

我把他們的需求整理成清單：

```
核心功能：
1. 記錄收支
2. 分類管理
3. 查看統計
4. 預算提醒
5. 資料匯出

使用者故事：
- 小美想要快速記帳
- 老陳想要了解消費模式
- 阿威想要控制預算
- 小李想要整理報稅資料
```

設計架構：畫出藍圖

在寫第一行程式碼之前，我們要先設計架構。

就像蓋房子要先有設計圖，寫程式也要先規劃結構。

```
記帳程式架構：

資料層：
├── 帳目資料（日期、金額、類別、備註）
├── 類別資料（飲食、交通、娛樂...）
└── 預算資料（每月預算上限）
```

```
功能層：
├── 新增帳目
├── 查詢帳目
├── 統計分析
├── 預算管理
└── 資料匯出

介面層：
├── 主選單
├── 輸入介面
└── 報表顯示
```

「原來要考慮這麼多！」阿威驚訝地說。

是的，這就是為什麼專案不同於練習題。練習題是單一功能，專案是完整系統。

第一步：資料結構

```python
import json
import datetime
from typing import Dict, List

class 記帳本:
    def __init__(self):
        self.帳目 = []
        self.類別 = [
            "飲食", "交通", "購物", "娛樂",
            "居住", "醫療", "教育", "其他"
        ]
        self.每月預算 = 30000
        self.載入資料()

    def 載入資料(self):
        """從檔案讀取之前的記錄"""
        try:
            with open("記帳資料.json", "r", encoding="utf-8") as f:
                資料 = json.load(f)
                self.帳目 = 資料.get("帳目", [])
                self.每月預算 = 資料.get("預算", 30000)
        except FileNotFoundError:
            print("第一次使用，歡迎！")

    def 儲存資料(self):
        """將資料存到檔案"""
        資料 = {
            "帳目": self.帳目,
            "預算": self.每月預算
        }
```

```
        with open("記帳資料.json", "w", encoding="utf-8") as f:
            json.dump(資料, f, ensure_ascii=False, indent=2)
```

「為什麼要儲存資料？」小美問。

「因為你不會想每次打開程式都要重新輸入所有記錄吧？」

持久化——讓資料在程式關閉後還能保存，這是真實專案的必備功能。

核心功能：新增帳目

```
def 新增支出(self, 金額: float, 類別: str, 備註: str = ""):
    """記錄一筆支出"""
    if 類別 not in self.類別:
        print(f"類別 '{類別}' 不存在，已歸類為『其他』")
        類別 = "其他"

    帳目 = {
        "日期": datetime.datetime.now().strftime("%Y-%m-%d"),
        "時間": datetime.datetime.now().strftime("%H:%M"),
        "類型": "支出",
        "金額": 金額,
        "類別": 類別,
        "備註": 備註
    }

    self.帳目.append(帳目)
    self.儲存資料()

    # 檢查預算
    self.檢查預算()

    print(f"✓ 已記錄：{類別} -${金額}")

def 新增收入(self, 金額: float, 來源: str = "薪資"):
    """記錄一筆收入"""
    帳目 = {
        "日期": datetime.datetime.now().strftime("%Y-%m-%d"),
        "時間": datetime.datetime.now().strftime("%H:%M"),
        "類型": "收入",
        "金額": 金額,
        "類別": 來源,
        "備註": ""
    }

    self.帳目.append(帳目)
    self.儲存資料()
```

```
    print(f"✓ 已記錄：{來源} +${金額}")
```

當小美第一次成功記錄一筆支出時，她笑了：「這是我寫的程式在記我的帳！」

那種創造的成就感，是無法形容的。

統計分析：讓數據說話

```
def 本月統計(self):
    """顯示本月收支統計"""
    本月 = datetime.datetime.now().strftime("%Y-%m")

    本月帳目 = [
        帳目 for 帳目 in self.帳目
        if 帳目["日期"].startswith(本月)
    ]

    總收入 = sum(
        帳目["金額"] for 帳目 in 本月帳目
        if 帳目["類型"] == "收入"
    )

    總支出 = sum(
        帳目["金額"] for 帳目 in 本月帳目
        if 帳目["類型"] == "支出"
    )

    餘額 = 總收入 - 總支出

    print(f"\n📊 {本月} 月統計")
    print("=" * 40)
    print(f"總收入：${總收入:,.0f}")
    print(f"總支出：${總支出:,.0f}")
    print(f"餘額：  ${餘額:,.0f}")

    # 支出分類統計
    分類統計 = {}
    for 帳目 in 本月帳目:
        if 帳目["類型"] == "支出":
            類別 = 帳目["類別"]
            分類統計[類別] = 分類統計.get(類別, 0) + 帳目["金額"]

    if 分類統計:
        print("\n📈 支出分類：")
        for 類別, 金額 in sorted(分類統計.items(), key=lambda x: x[1],
reverse=True):
            百分比 = (金額 / 總支出) * 100
```

```
            進度條 = "■" * int(百分比 / 5)
            print(f"  {類別:  <4} ${金額:>8,.0f} {進度條} {百分比:.1f}%")
```

老陳看著統計結果驚呼：「原來我40%的錢都花在飲食上！」

數據不會說謊。當你看到真實的數字，你才知道錢去了哪裡。

預算管理：自我約束

```
def 設定預算(self, 金額: float):
    """設定每月預算"""
    self.每月預算 = 金額
    self.儲存資料()
    print(f"✓ 每月預算已設定為：${金額:,.0f}")

def 檢查預算(self):
    """檢查是否超出預算"""
    本月 = datetime.datetime.now().strftime("%Y-%m")

    本月支出 = sum(
        帳目["金額"] for 帳目 in self.帳目
        if 帳目["日期"].startswith(本月) and 帳目["類型"] == "支出"
    )

    使用率 = (本月支出 / self.每月預算) * 100
    剩餘預算 = self.每月預算 - 本月支出

    if 使用率 > 100:
        print(f"⚠️ 警告：本月已超支 ${-剩餘預算:,.0f}！")
    elif 使用率 > 80:
        print(f"⚠️ 注意：預算已使用 {使用率:.0f}%，剩餘 ${剩餘預算:,.0f}")
    elif 使用率 > 50:
        print(f"💰 預算使用 {使用率:.0f}%，剩餘 ${剩餘預算:,.0f}")
```

阿威設定了娛樂預算3000元。當系統提醒他只剩500元時，他猶豫了一下，放棄了原本想看的電影。

「程式在幫我省錢！」他說。

使用者介面：讓程式活起來

```python
def 主選單(self):
    """顯示主選單"""
    while True:
        print("\n" + "=" * 40)
        print("💼 個人記帳本")
        print("=" * 40)
        print("1. 記錄支出")
        print("2. 記錄收入")
        print("3. 查看本月統計")
        print("4. 查看歷史記錄")
        print("5. 設定預算")
        print("6. 匯出報表")
        print("0. 離開")

        選擇 = input("\n請選擇功能：")

        if 選擇 == "1":
            self.輸入支出()
        elif 選擇 == "2":
            self.輸入收入()
        elif 選擇 == "3":
            self.本月統計()
        elif 選擇 == "4":
            self.顯示歷史()
        elif 選擇 == "5":
            self.輸入預算()
        elif 選擇 == "6":
            self.匯出報表()
        elif 選擇 == "0":
            print("再見！記得持續記帳哦！")
            break
        else:
            print("無效的選擇，請重新輸入")

def 輸入支出(self):
    """互動式輸入支出"""
    print("\n📝 記錄支出")
    print("-" * 20)

    # 顯示類別選項
    print("類別選擇：")
    for i, 類別 in enumerate(self.類別, 1):
        print(f"  {i}. {類別}")

    類別索引 = input("請選擇類別（輸入數字）：")
    try:
        類別 = self.類別[int(類別索引) - 1]
    except:
        類別 = "其他"

    金額 = float(input("金額：$"))
    備註 = input("備註（可選）：")

    self.新增支出(金額, 類別, 備註)
```

介面讓程式從冰冷的代碼變成可以互動的工具。

小李說：「雖然沒有漂亮的圖形界面，但這個文字界面反而讓我更專注。」

除錯與改進：專案的成長

第一版完成後，大家開始使用。然後問題來了。

「如果輸入錯了怎麼刪除？」小美問。「能不能看特定月份的統計？」老陳問。「可以設定不同類別的預算嗎？」阿威問。

這就是真實專案的生命週期——永遠有新需求，永遠在改進。

```
def 刪除最後一筆(self):
    """刪除最後一筆記錄"""
    if not self.帳目:
        print("沒有記錄可以刪除")
        return

    最後一筆 = self.帳目.pop()
    self.儲存資料()

    print(f"✓ 已刪除：{最後一筆['日期']} {最後一筆['類別']} $${最後一筆['金額']}")

def 搜尋帳目(self, 關鍵字: str):
    """搜尋特定帳目"""
    結果 = [
        帳目 for 帳目 in self.帳目
        if 關鍵字 in 帳目.get("備註", "") or 關鍵字 in 帳目["類別"]
    ]

    if 結果:
        for 帳目 in 結果:
            print(f"{帳目['日期']} {帳目['類別']} $${帳目['金額']} {帳目.get('備註',
''')}")
    else:
        print(f"找不到包含 '{關鍵字}' 的記錄")
```

每個新功能都讓專案更完整，也讓你的程式能力更進步。

部署與分享：讓別人使用你的作品

當記帳程式越來越完善，朋友們開始詢問能不能也用。

這時候你要學會「部署」——讓程式能在別人的電腦上運行。

```
# 打包成執行檔
# requirements.txt
pandas==1.5.0
matplotlib==3.6.0

# 安裝說明
"""
安裝步驟：
1. 安裝Python 3.8以上版本
2. 執行：pip install -r requirements.txt
3. 執行：python 記帳本.py

或者使用打包工具：
pyinstaller --onefile 記帳本.py
"""
```

當小美把程式分享給她媽媽，而她媽媽真的開始用它記帳時，小美激動地說：

「我寫的程式在幫助別人！這種感覺太奇妙了！」

專案的意義

完成這個記帳程式後，每個人都有不同的收穫。

小美：「我終於理解什麼是『程式設計』了。不只是寫code，而是解決問題。」

阿威：「原來創造數位產品不一定要很複雜，簡單但實用就很好。」

老陳：「寫程式就像蓋房子，一磚一瓦，慢慢就有了形狀。」

小李：「最大的成就感是——我每天都在用自己寫的程式。」

寫在第六章的最後

你的第一個專案不需要改變世界，只需要解決一個真實的問題。

可能是：

- 整理照片的小工具
- 背單字的程式
- 記錄運動的App
- 管理待辦事項的系統

重要的不是專案有多厲害，而是你完成了它。

從零到一，從無到有，你創造了一個原本不存在的東西。

這就是程式設計的魔力——把想法變成現實。

記住今天的感覺，記住第一次看到自己的程式運行時的興奮。

因為這只是開始。

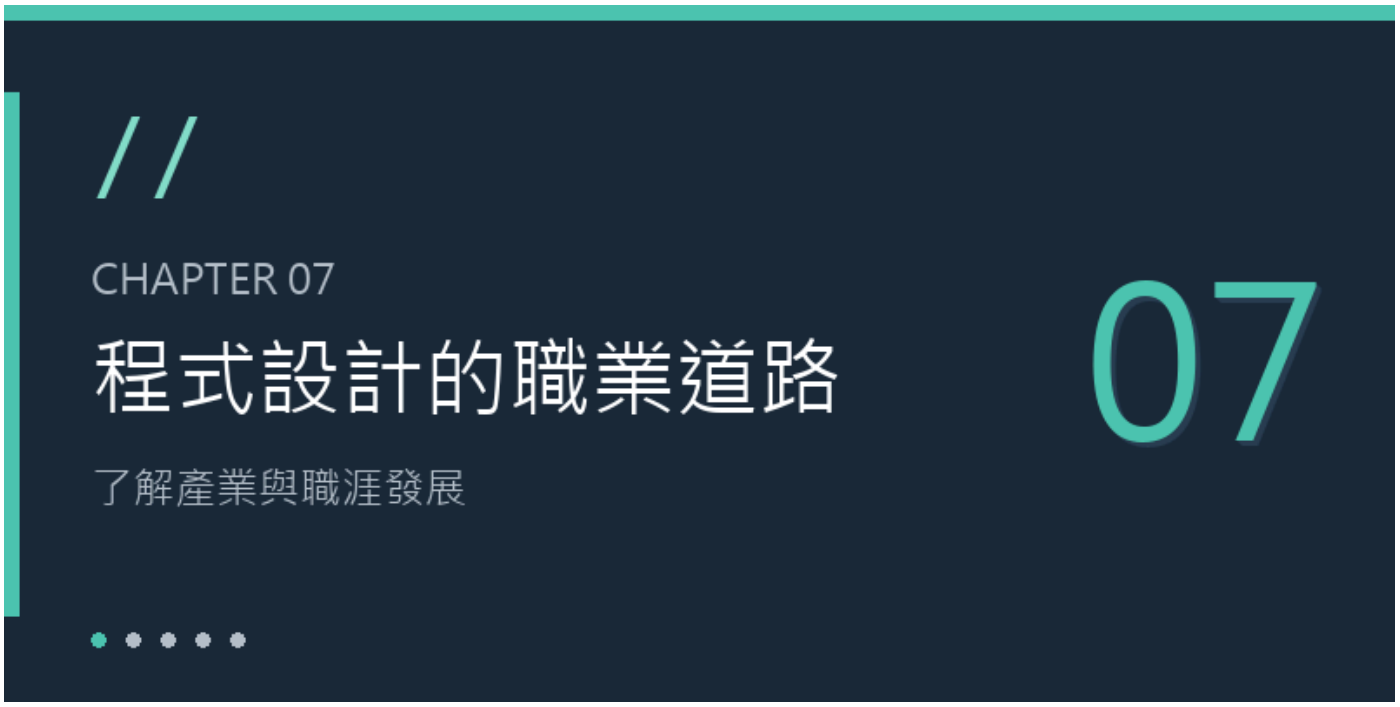
下一章，我們要談談如果你想把這份興趣變成職業，路該怎麼走。

但無論你最終是否成為職業工程師，今天你已經證明了——你能創造。

而能創造的人，永遠不會被時代淘汰。

「*The best way to predict the future is to invent it.*」 *- Alan Kay*

預測未來的最好方法，就是創造它。 而你，已經開始創造了。



第7章：程式設計的職業道路

> 「如果我真的想轉職成為工程師，機會大嗎？」
>
> 小美完成記帳程式後，第一次認真思考這個問題。她35歲，沒有資工背景，但她已經能寫出實用的程式了。

咖啡廳裡的職涯討論

週六下午，我們約在常去的咖啡廳。

小美、阿威、老陳、小李都來了。還有一位新朋友——凱文，一位在科技公司工作五年的前端工程師。

「我也是轉職的，」凱文說，「28歲才開始學程式，之前是會計。」

大家的眼睛都亮了起來。

凱文繼續：「轉職不是問題，問題是你要知道自己在做什麼。工程師不是一種職業，而是很多種職業的統稱。」

他在餐巾紙上畫了一張圖...

工程師的光譜

```
使用者能看到的 ← - - - - - - - - → 使用者看不到的

前端工程師            全端工程師            後端工程師
    |                    |                    |
設計實現              兩者都做              邏輯處理
使用者體驗            獨立開發              資料管理
視覺呈現              快速迭代              系統架構

        行動開發工程師
              |
        iOS / Android
         原生 App

                    DevOps 工程師
                          |
                    部署與維運
                    自動化流程

                              資料工程師
                                    |
                              資料處理
                              ETL 流程

                                        AI/ML 工程師
                                              |
                                        機器學習
                                        深度學習
```

「每個方向需要的技能不同，工作內容不同，連個性需求都不同。」凱文說。

前端工程師：網頁的藝術家

阿威最感興趣的是前端。

「前端工程師就是把設計變成真實網頁的人，」凱文解釋，「你要懂美感，也要懂程式。」

日常工作：

- 早上開 standup meeting，討論今天要做什麼
- 實作設計師的設計稿
- 處理使用者互動（點擊、滑動、輸入）

- 優化網頁載入速度
- 確保在不同瀏覽器都正常顯示
- 與後端工程師協調 API 介面

需要的技能：

```
// 三劍客
HTML：網頁的骨架
CSS：網頁的外觀
JavaScript：網頁的行為

// 現代框架
React / Vue / Angular：其中一個

// 工具
Git：版本控制
Webpack：打包工具
npm：套件管理
```

適合的個性：

- 注重細節（1px 的差異都要在意）
- 有美感
- 喜歡即時看到成果
- 能接受快速變化（前端技術更新很快）

阿威聽完說：「這簡直是為我量身定做的！我本來就是設計師，會程式後就能自己實現設計了。」

後端工程師：系統的建築師

老陳對後端更有興趣。

「後端是處理商業邏輯的地方，」凱文說，「使用者看不到，但一切運作都靠它。」

日常工作：

- 設計資料庫結構
- 撰寫 API 介面
- 處理商業邏輯（訂單、付款、庫存）
- 優化查詢效能

- 確保資料安全
- 處理大流量（如果網站很熱門）

需要的技能：

```
# 語言（選一個深入）
Python + Django/Flask
JavaScript + Node.js
Java + Spring
Go

# 資料庫
SQL：關聯式資料庫
NoSQL：MongoDB、Redis

# 概念
RESTful API
微服務架構
快取策略
資安基礎
```

適合的個性：

- 邏輯思維強
- 喜歡解決複雜問題
- 注重系統穩定性
- 能處理抽象概念

老陳點頭：「銀行系統就是大型的後端，我懂這個邏輯。」

全端工程師：瑞士刀

小李對全端有興趣。

「全端就是前後端都會，」凱文說，「通常在新創公司很受歡迎，因為一個人能做很多事。」

優點：

- 能獨立完成整個專案
- 了解系統全貌
- 轉職彈性大

- 新創公司愛用

挑戰：

- 要學的東西很多
- 難以在單一領域深入
- 技術更新壓力大

「但是，」凱文提醒，「不要一開始就想當全端。先專精一端，再學另一端。」

資料工程師：數據管道工

小美發現自己對數據特別有興趣。

「資料工程師負責確保資料能順暢流動，」凱文解釋，「從收集、清理、轉換到儲存。」

日常工作：

- 建立資料管道（ETL）
- 清理髒資料
- 設計資料倉儲
- 優化查詢效能
- 確保資料品質

薪資往往很高，因為：

- 需求大（大數據時代）
- 會的人相對少
- 直接影響商業決策

小美眼睛發亮：「我每天在Excel裡做的事，原來可以自動化成這樣！」

轉職的真實路徑

「好了，了解方向後，要怎麼轉職？」小美問。

凱文分享了他的經驗：

第0-3個月：打基礎

```
學習內容：
- 程式語言基礎
- 資料結構與演算法基礎
- Git 版本控制

每天投入：2-3小時
重點：建立程式思維
```

第4-6個月：選定方向

```
前端路線：
- 深入 JavaScript
- 學一個框架（React/Vue）
- 做3個小專案

後端路線：
- 深入一個後端語言
- 學資料庫
- 寫 RESTful API
```

第7-9個月：作品集

```
關鍵任務：
- 完成1-2個完整專案
- 放上 GitHub
- 寫技術部落格
- 參加技術社群
```

第10-12個月：求職準備

```
面試準備：
- 刷 LeetCode（基礎題就好）
- 準備作品集簡報
- 模擬面試
- 投遞履歷
```

「一年？」阿威有點驚訝。

「如果你全職學習，半年也可能。如果只能晚上學，可能要兩年。」凱文說，「重點是持續和累積。」

第一份工作的真相

「第一份工程師工作最難找，」凱文坦白說，「但一旦有了一年經驗，機會就完全不同了。」

新手的機會在哪？

1. 新創公司

- 願意給新手機會
- 學習曲線陡峭（學很快）
- 可能要一人多工
- 薪水可能較低

2. 接案/自由工作

- 累積作品集
- 建立人脈
- 收入不穩定
- 需要自律

3. 大公司的約聘

- 有機會轉正
- 能學到規範
- 競爭激烈

4. 內部轉職

- 在現有公司轉技術職

- 已有領域知識
- 風險較低

小美若有所思：「我可以先在公司內部推動數位轉型，邊做邊學...」

薪資的現實

大家最關心的問題來了。

凱文打開了一個薪資網站：

```
台灣市場（2024年）：

Junior（0-2年）
前端：40-60萬/年
後端：45-65萬/年
全端：45-65萬/年

Mid-level（2-5年）
前端：60-90萬/年
後端：65-100萬/年
全端：70-100萬/年

Senior（5年以上）
前端：90-150萬/年
後端：100-180萬/年
全端：100-180萬/年

外商/新創可能更高
遠端工作機會增加
```

「但是，」凱文提醒，「不要只看錢。第一份工作重點是學習機會。」

工程師的日常真相

「我想知道真實的一天是怎樣的。」老陳問。

凱文打開他的行事曆：

```
09:00 - 到公司，看昨天的 code review 評論
09:30 - 每日站會，15分鐘
```

```
09:45 - 寫程式（新功能開發）
11:00 - 幫同事 debug
12:00 - 午餐
13:30 - 寫程式（繼續早上的）
15:00 - 開會討論下週規劃
16:00 - Code review 其他人的程式
17:00 - 寫文件/測試
18:30 - 下班（有時候）

實際上：
- 30% 寫新程式
- 30% 改別人的程式/debug
- 20% 開會討論
- 20% 寫文件/測試
```

「不是一直在寫程式？」阿威驚訝。

「溝通和協作佔很大部分，」凱文說，「這就是為什麼軟實力也很重要。」

持續學習的宿命

「當工程師最大的挑戰，」凱文正色說，「是你永遠在學習。」

```
5年前的熱門技術 → 現在可能過時
現在的新技術 → 5年後可能成主流

例如：
jQuery（曾經必學）→ 現在很少用
React（5年前的新秀）→ 現在的主流
AI 輔助編程 → 未來的趨勢？
```

「這不是很累嗎？」小美擔心。

「換個角度想，」凱文笑了，「你永遠不會無聊。而且，基礎觀念不變，變的只是工具。」

不當工程師也很好

「我想說一件事，」凱文認真地說，「不是每個學程式的人都要當工程師。」

程式能力 **+** 領域知識 **=** 獨特價值

- 小美：行銷 + 程式 = 數據行銷專家
- 阿威：設計 + 程式 = 創意技術專家
- 老陳：金融 + 程式 = FinTech 專家
- 小李：業務 + 程式 = 銷售自動化專家

「會程式的行銷人員，比純工程師更了解行銷需求。」「會程式的設計師，能創造別人做不出的互動體驗。」「會程式的金融人，能開發別人想不到的交易策略。」

寫在第七章的最後

咖啡廳的討論持續到傍晚。

離開前，凱文說了一段話：

「五年前，我是個每天對著報表的會計。現在，我在開發影響百萬用戶的產品。不是因為我特別聰明，而是因為我開始了，並且堅持下去。」

「轉職不是終點，而是另一個起點。重要的不是你幾歲開始，而是你願意投入多少。」

「記住，科技業缺的不是會寫程式的人，而是能解決問題的人。你的過去不是包袱，而是你的獨特優勢。」

小美若有所思地說：「我不確定我會不會成為全職工程師，但我確定，學程式改變了我看問題的方式。」

阿威興奮地說：「我要開始準備作品集了！」

老陳淡定地說：「慢慢來，我先把基礎打好。」

小李已經在查線上課程了。

每個人都有自己的步調，自己的目標。

而這，正是最好的開始。

「*The expert in anything was once a beginner.*」

任何領域的專家，都曾經是初學者。 你的程式之路，由你定義。

第8章：學習資源與社群

> 「網路上資源太多了，我反而不知道從哪裡開始。」
>
> 阿威打開瀏覽器，書籤列裡存了上百個程式學習網站，但他一個都沒有認真看完。

資訊焦慮的時代

我們生活在一個矛盾的時代。

從來沒有一個時代，學習資源如此豐富——YouTube上有無數免費教學、GitHub上有海量開源專案、Medium上有數不清的技術文章。

但也從來沒有一個時代，學習者如此焦慮——怕選錯方向、怕落後潮流、怕浪費時間。

「我光是選要看哪個教學，就花了一個禮拜。」阿威苦笑。

這就是選擇的詛咒。當選項太多，我們反而無法行動。

讓我告訴你一個秘密：你不需要看完所有資源。你只需要找到適合你的那一個，然後堅持下去。

免費資源的寶庫

YouTube：視覺學習者的天堂

小李是視覺學習者，他需要看到畫面才能理解。

推薦頻道：

freeCodeCamp

- 特色：超長但完整的教學（動輒4-8小時）
- 適合：想要一次學透徹的人
- 小李的體驗：「雖然很長，但講得很細，適合週末慢慢看。」

Traversy Media

- 特色：實作導向，跟著做就會
- 適合：想快速上手的人
- 阿威的最愛：「Brad的聲音很舒服，而且專案都很實用。」

彭彭的課程（中文）

- 特色：中文教學，適合初學者
- 適合：英文不太好的朋友
- 小美的救星：「終於有人用中文解釋那些概念了！」

但YouTube有個陷阱——你很容易陷入「教學地獄」，一直看影片卻不動手寫。

記住：看一小時，練習兩小時。

GitHub：程式碼的圖書館

老陳第一次上GitHub時嚇到了：「這麼多程式碼，都是免費的？」

是的，這就是開源的力量。

如何利用**GitHub**學習：

1. 讀優秀的程式碼

```
搜尋技巧：
- 加上 "awesome" 找精選列表
- 按星星數排序找熱門專案
- 看 Trending 了解潮流
```

2. 從**README**開始 好的專案都有詳細的README，告訴你：

- 這個專案做什麼
- 如何安裝
- 如何使用
- 如何貢獻

3. 追蹤學習路徑 很多人整理了學習路徑：

- developer-roadmap（最完整的技能樹）
- free-programming-books（免費書籍大全）
- project-based-learning（用專案學習）

老陳現在每天早上都會看GitHub Trending：「就像看報紙，了解技術圈在關注什麼。」

線上練習平台

Codecademy：互動式學習

```
優點：
- 左邊教學，右邊寫程式
- 即時回饋
- 循序漸進

缺點：
- 免費版內容有限
- 太過引導，缺乏思考

小美的評價：
「適合第一週，讓你有信心。
但要真的學會，還是要自己寫專案。」
```

HackerRank / LeetCode：面試準備

```
時機：
不要太早開始刷題！
先把基礎打好，
```

```
準備面試時再來。

策略：
- 從Easy開始
- 重質不重量
- 理解思路比背答案重要
```

付費課程的價值

「免費資源這麼多，為什麼還要付費？」小美曾經這樣問。

直到她買了第一堂線上課程，她才明白。

Udemy：課程超市

```
優點：
- 結構完整
- 有作業和專案
- 可以問問題
- 常常特價（$300-500台幣）

選課技巧：
- 看評價（4.5星以上）
- 看更新日期（一年內）
- 看課程大綱（是否符合需求）
- 看預覽影片（講師風格）

阿威的經驗：
「我買了一堂React課程，
講師從零開始教，
最後做出一個完整的電商網站。
這種系統性是YouTube很難找到的。」
```

Coursera：學院派

```
特色：
- 大學教授授課
- 理論扎實
- 有作業批改
- 可以拿證書

適合：
```

```
- 想要扎實基礎
- 需要證書
- 喜歡學院風格

老陳的選擇：
「我上了Princeton的演算法課，
雖然很硬，但真的懂了原理。」
```

台灣的選擇

六角學院

- 專注前端
- 中文教學
- 有助教系統
- 社群活躍

Hahow

- 課程多元
- 價格親民
- 終身觀看

Alpha Camp

- 轉職導向
- 有mentor制度
- 就業輔導

社群的力量

學程式最孤獨的不是遇到bug，而是沒人可以討論。

Facebook社團

台灣的程式社團：

「JavaScript.tw」

- 成員：3萬+
- 氣氛：友善，新手可以問問題
- 小美：「我的第一個問題就是在這裡得到解答的。」

「Python Taiwan」

- 成員：2萬+
- 特色：定期有實體聚會

「前端開發者」

- 討論熱烈
- 資訊更新快
- 阿威每天必看

參與技巧：

1. 先搜尋，可能有人問過
2. 問問題要提供足夠資訊
3. 得到幫助要說謝謝
4. 有能力時回答別人

Discord / Slack

即時討論的好地方。

凱文推薦：「加入一個活躍的Discord，就像有了一個線上讀書會。」

```
好的社群特徵：
- 有明確規則
- 管理員活躍
- 成員互助
- 定期活動
```

實體聚會

Meetup

台北每週都有技術聚會：

- ReactJS Taiwan
- Python Taipei
- GDG Taiwan

「第一次參加很緊張，」小李回憶，「但大家都很友善，還交到了朋友。」

好處：

- 面對面交流
- 拓展人脈
- 了解業界動態
- 免費披薩（笑）

部落格與寫作

「寫技術文章？我還在學，能寫什麼？」阿威疑惑。

這是最大的誤解。

為什麼要寫

凱文分享：「寫文章不是為了教別人，而是為了整理自己的思緒。」

寫作的好處：

1. 加深理解（教是最好的學）
2. 建立個人品牌
3. 記錄學習歷程
4. 幫助other beginners

寫什麼

```
初學者可以寫：
- 今天學了什麼
- 解決了什麼bug
- 讀了什麼好文章
- 做了什麼小專案
```

不用寫教學，
寫學習筆記就很有價值。

在哪寫

Medium

- 介面漂亮
- 容易被發現
- 可以賺點錢

dev.to

- 程式社群
- 技術氛圍濃
- 國際曝光

iT邦幫忙

- 台灣最大技術社群
- 鐵人賽很有挑戰性
- 中文友善

小美開始寫部落格後：「寫下來才發現自己懂了多少，不懂多少。」

播客與新媒體

通勤時間也能學習。

推薦播客

JavaScript Jabber 適合：通勤聽，了解趨勢

CodeNewbie 適合：初學者，聽別人的故事

星箭廣播（中文） 適合：了解科技趨勢

老陳：「每天通勤一小時，現在變成我的學習時間。」

Twitter/X 追蹤

追蹤對的人，timeline就是最好的學習資源。

```
值得追蹤：
- Dan Abramov (React核心)
- Addy Osmani (Google Chrome)
- Sara Soueidan (CSS大師)

追蹤原則：
- 質量而非數量
- 實作者而非KOL
- 多元觀點
```

學習資源的斷捨離

Marie Kondo說：只留下讓你怦然心動的物品。

學習資源也該這樣。

資源評估矩陣

```
        有用 ←———→ 沒用
花時間↑
      │    A區       B區
      │  深入學習   立刻刪除
      │
      │    C區       D區
省時間↓  快速參考   偶爾看看
```

個人學習系統

```
核心資源（1-2個）：
主要學習來源，堅持看完

輔助資源（3-5個）：
遇到問題時查詢
```

靈感資源（隨意）：
閒暇時瀏覽，保持熱情

阿威整理後：「我把書籤從300個減到30個，反而學得更多了。」

寫在第八章的最後

資源和社群都是工具，真正的學習發生在你動手的時候。

最好的資源不是最貴的、最新的、最熱門的，而是你真的會用的。

最好的社群不是最大的、最活躍的、最厲害的，而是你覺得舒服的。

記住：

- 一個看完的課程，勝過十個收藏的連結
- 一個寫完的專案，勝過百個看過的教學
- 一個真心的朋友，勝過千個社團成員

小美現在有自己的學習節奏：早上看文檔、下午寫程式、晚上逛社群。

阿威找到了自己的學習風格：看影片入門、做專案深入、寫文章總結。

老陳建立了學習習慣：每天一小時、每週一專案、每月一分享。

小李加入了學習社群：線上討論、線下聚會、互相成長。

你呢？你的學習系統是什麼？

下一章，我們要談談學習路上最重要的東西——心態。

因為資源會過時，技術會更新，但正確的心態會陪你走得更遠。

「*The capacity to learn is a gift; The ability to learn is a skill; The willingness to learn is a choice.*」

學習的能力是天賦， 學習的技巧是技能， 學習的意願是選擇。

而你已經做出了選擇。

第9章：學習程式的正確心態

> 「我已經卡在這個bug三個小時了，是不是我真的不適合寫程式？」
>
> 深夜兩點，阿威盯著螢幕，眼睛布滿血絲。錯誤訊息還是一樣，程式還是跑不動。

深夜的崩潰

那是一個星期三的深夜。

阿威想要完成一個作品集網站，展示他的設計作品。本來以為很簡單的功能——點擊圖片放大顯示——就是做不出來。

Google了無數解法，Stack Overflow看了幾十篇，程式碼改了又改，還是不行。

「為什麼別人的程式碼都能跑，就我的不行？」

他關上筆電，走到陽台。城市的夜景很美，但他沒有心情欣賞。

「也許我真的沒有寫程式的天份。」

這個念頭不是第一次出現。每次遇到解不開的bug，每次看到別人輕鬆寫出複雜功能，每次面對新技術的學習曲線，這個聲音就會在耳邊響起。

如果你也曾經有過這樣的時刻，我想告訴你——這很正常。

每個工程師都經歷過。包括那些你以為的「天才」。

---

bug是最好的老師

隔天早上，阿威帶著黑眼圈來找我。

「我想放棄了。」他說。

我請他把程式碼打開，一起看看。

五分鐘後，我們找到了問題——一個拼寫錯誤。`getElementById` 寫成了 `getElementById`（ID大寫了）。

就這樣。三個小時的痛苦，源於兩個字母的大小寫。

阿威愣住了，然後苦笑：「就這樣？」

「就這樣。」

「我覺得自己好蠢。」

「不，」我說，「你剛剛學到了一課——JavaScript區分大小寫。而且我保證，你這輩子都不會再犯這個錯誤。」

這就是bug的價值。每個bug都是一個偽裝的老師：

```
Bug教會我們的事：

拼寫錯誤 → 細心的重要性
邏輯錯誤 → 思考的嚴謹性
效能問題 → 優化的必要性
安全漏洞 → 防禦的意識

每解決一個bug，
你就升級一點。
```

凱文分享了他的經驗：「我有一個『bug日記』，記錄每個遇到的問題和解法。現在回頭看，那些曾經讓我崩潰的bug，都成了我的養分。」

冒牌者症候群

小美最近加入了一個程式學習群組。

「裡面的人都好厲害，」她說，「討論的東西我都聽不懂。我是不是太弱了？」

這叫「冒牌者症候群」（Impostor Syndrome）——覺得自己不夠格，隨時會被發現是「冒牌貨」。

真相是什麼？

我打開了一個調查結果：

- 70%的工程師經常感到自己能力不足
- 58%的資深工程師仍然有冒牌者症候群
- 即使是技術主管，也常常懷疑自己

「連資深工程師都這樣？」小美驚訝。

是的。因為技術永遠在進步，永遠有你不知道的東西。

如何面對冒牌者症候群

1. 承認你的成長

小美三個月前連變數是什麼都不知道，現在已經能寫自動化程式了。

「列出你學會的東西，」我建議，「你會驚訝自己進步了多少。」

```
三個月前的我：
- 不知道變數是什麼
- 沒寫過一行程式碼
- 看到終端機就害怕

現在的我：
- 能寫Python程式
- 完成了5個小專案
- 幫公司自動化了3個流程
```

2. 理解知識的層次

```
程式知識像一座冰山：

水面上（看得見的）：
- 別人展示的成果
- 技術文章的精華
- 開源專案的成品

水面下（看不見的）：
- 無數的失敗嘗試
- 數不清的bug除錯
- 漫長的學習過程
```

你看到的「厲害」，都是冰山一角。

3. 專注於自己的進度

「不要和別人比，要和昨天的自己比。」

老陳深有體會：「我50歲才開始學，當然比不過20歲的年輕人。但那又怎樣？我只要比昨天的我進步就好。」

學習高原期

學習曲線不是一條斜直線，而是階梯狀的。

```
理想的學習曲線：
能力 ↑
      /////
     /////
    /////
   /////
時間 →

實際的學習曲線：
能力 ↑
         _________/
        |
    ____/
   |
時間 →
```

那些平坦的部分，就是「高原期」。

小李最近就在高原期：「我感覺自己沒在進步，每天寫的程式碼都差不多。」

這是最危險的時期，很多人在這裡放棄。

突破高原期的方法

1. 改變學習內容

如果你一直在寫CRUD（增刪改查），試試：

- 學習新的框架
- 挑戰不同類型的專案
- 深入研究原理

2. 提高難度

阿威突破高原期的方法：「我給自己設限制，比如不用框架實現功能，或者優化程式效能到極致。」

3. 教學相長

「開始寫部落格後，我發現自己理解更深了。」小美說。

教別人是最好的學習方式。

完美主義的陷阱

阿威是完美主義者。

他的第一個專案，改了三個月還沒上線。

「還不夠好，」他總是這麼說，「再改改。」

直到有一天，他發現類似的網站已經有五個了，而他的還在本機裡。

Done is better than perfect

Facebook辦公室有句標語：「Done is better than perfect」（完成比完美更重要）。

> 完美主義的代價：

```
拖延上線 → 錯失機會
過度優化 → 浪費時間
害怕批評 → 不敢分享
追求完美 → 永遠無法完成
```

正確的做法：

1. 先求有，再求好
2. 快速迭代，持續改進
3. 接受「夠好」的標準
4. 從回饋中學習

凱文的建議：「設定期限。時間到了就發布，不管完不完美。你會發現，用戶在意的往往不是你糾結的細節。」

持續學習的智慧

「學程式最累的是，永遠學不完。」小美嘆氣。

確實，新技術層出不窮：

- 昨天流行React
- 今天流行Vue
- 明天可能是Svelte

但這裡有個秘密：你不需要學所有東西。

學習的優先順序

```
核心知識（必學）：
- 程式邏輯
- 資料結構基礎
- 問題解決能力
├─ 這些10年都不會變

工作需要（選學）：
- 公司使用的技術
- 產業標準工具
├─ 這些能幫你賺錢

興趣導向（想學）：
- 新奇的技術
```

```
- 好玩的專案
├─ 這些讓你保持熱情
```

FOMO vs JOMO

FOMO（Fear of Missing Out）：害怕錯過 JOMO（Joy of Missing Out）：享受錯過

老陳的哲學：「我不追新技術，我追穩定技術。當大家都在學最新框架時，我在深入研究JavaScript基礎。結果？框架換了三輪，我的基礎知識還是有用。」

失敗是功能，不是bug

每個成功的工程師都有失敗的故事。

凱文分享了他的：

- 第一個專案，資料庫設計錯誤，重寫三次
- 第一次面試，連FizzBuzz都寫不出來
- 第一個上線產品，被駭客攻擊

「但每次失敗，都讓我變強。」

重新定義失敗

```
def 學習程式(嘗試):
    if 成功:
        return 獲得經驗
    else:
        return 獲得教訓

# 注意：沒有真正的失敗
# 只有不同形式的學習
```

失敗的價值：

- 知道什麼不可行
- 理解問題的複雜性
- 建立心理韌性

- 累積除錯經驗

找到你的為什麼

當學習變得困難，當bug解不開，當新技術讓你頭痛，記得你的「為什麼」。

小美的為什麼：「我想要更多時間陪家人，不想把時間浪費在重複工作上。」

阿威的為什麼：「我想創造美的東西，而且是能動的、有互動的。」

老陳的為什麼：「我想證明50歲也能學新東西。」

小李的為什麼：「我想要自由，能在任何地方工作。」

你的為什麼是什麼？

寫下來，貼在螢幕旁。當你想放棄時，看看它。

慶祝小勝利

我們太習慣關注還不會的，忘記慶祝已經會的。

建立慶祝儀式：

- 解決一個bug → 伸個懶腰，smile
- 完成一個功能 → 喝杯好咖啡
- 上線一個專案 → 和朋友分享

「我開始記錄『成就日記』，」阿威說，「每天寫下一件進步的事，哪怕很小。」

```
今天的小勝利：
✓ 理解了閉包的概念
✓ 修復了一個困擾兩天的bug
✓ 幫同事解決了問題
✓ 程式碼比昨天寫得更簡潔
```

累積小勝利，就能堆出大成就。

寫在第九章的最後

學程式不只是學技術，更是一場心理修煉。

你會遇到：

- 讓你懷疑自己的bug
- 讓你焦慮的新技術
- 讓你沮喪的面試
- 讓你迷茫的選擇

但記住，這些都是過程，不是終點。

每個你admire的工程師，都走過同樣的路。他們不是沒有挫折，而是學會了與挫折共處。

正確的心態不是永遠正面，而是：

- 接受困難是常態
- 相信自己能成長
- 享受解決問題的過程
- 保持好奇心

最後，我想分享一句話，這是一位資深工程師告訴我的：

「寫程式就像種樹。你不會因為樹長得慢就懷疑自己不會種樹。你只是每天澆水，總有一天，你會看到它開花結果。」

你已經在澆水了。

繼續澆，保持耐心。

花會開的。

「*The master has failed more times than the beginner has even tried.*」

大師失敗的次數，比初學者嘗試的次數還多。

所以，繼續嘗試吧。 每一次嘗試，都讓你離大師更近一步。

第10章：從零到一的行動計畫

> 「我讀完了前面九章，很感動，很有動力，但...然後呢？」
>
> 你闔上書，看著窗外。心中有一團火被點燃，但不知道該往哪個方向燒。

最後一堂課

這是我們的最後一次見面。

小美、阿威、老陳、小李，還有你，我們圍坐在熟悉的咖啡廳裡。

桌上攤開著過去幾個月的筆記、程式碼、專案截圖。每個人的筆電上都有自己寫的程式在運行。

「三個月前，我連變數是什麼都不知道，」小美說，一邊展示她的記帳程式，「現在我的程式每天幫公司省下兩個小時。」

「我以為50歲學程式太晚了，」老陳笑著搖頭，「結果我現在是部門的Python達人。」

阿威打開他的作品集網站：「這全部都是我自己寫的。從設計到程式碼。」

小李正在優化他的爬蟲程式：「下個月我要去面試數據分析師了。」

他們都做到了。從零到一。

現在，輪到你了。

第一天：選擇你的武器

不要糾結太久。記住第二章說的——語言是工具，不是目的。

```
如果你是：
小美（行銷/業務）→ Python
理由：資料處理強大，學習曲線平緩

阿威（設計/創意）→ JavaScript
理由：網頁互動必備，視覺回饋即時

老陳（金融/分析）→ Python
理由：數據分析首選，函式庫豐富

小李（想轉職）→ JavaScript 或 Python
理由：工作機會多，社群資源豐富

還是不確定？→ Python
理由：最適合初學者的語言
```

今天的任務：

1. 安裝Python（去python.org）或Node.js（去nodejs.org）
2. 安裝VS Code（去code.visualstudio.com）
3. 寫下你的第一行程式碼：`print("我開始了")`

就這樣。第一天不要貪心。

第一週：建立習慣

習慣比天賦更重要。

我見過太多人第一天學8小時，然後消失三個月。別這樣。

每日**30**分鐘計畫：

```
週一：變數與資料型態
範例：姓名 = "你的名字"
      年齡 = 你的年齡
      print(f"我是{姓名}，今年{年齡}歲")

週二：條件判斷
範例：如果年齡 >= 18:
          print("成年人")
      否則:
          print("未成年")

週三：迴圈
範例：for 次數 in range(10):
          print(f"第{次數+1}次練習")

週四：函數
範例：def 問候(名字):
          return f"你好，{名字}！"

週五：綜合練習
任務：寫一個猜數字遊戲

週末：回顧與整理筆記
```

關鍵：每天都要碰程式碼，哪怕只有10分鐘。

第一個月：打地基

四週，四個里程碑。

第**1**週：基礎語法

- 目標：能寫出簡單的互動程式
- 專案：計算機、單位轉換器
- 心態：允許自己犯錯

第**2**週：資料處理

- 目標：能讀寫檔案，處理資料
- 專案：通訊錄管理、CSV處理
- 心態：享受自動化的快感

第**3**週：函數與模組

- 目標：能組織程式碼，重複使用

- 專案：待辦事項App
- 心態：追求簡潔優雅

第**4**週：第一個完整專案

- 目標：完成一個自己會用的工具
- 專案：個人記帳本（像第六章）
- 心態：完成比完美重要

月底檢視：

- 寫了多少行程式碼？
- 解決了什麼實際問題？
- 下個月想學什麼？

第一季：看見成長

三個月，足以改變一切。

第**2**個月：擴展技能

```
選擇你的方向：

網頁方向：
- HTML/CSS基礎
- JavaScript DOM操作
- 簡單的互動網頁

資料方向：
- pandas資料處理
- 簡單的資料視覺化
- Excel自動化

自動化方向：
- 檔案批次處理
- 網路爬蟲基礎
- 排程任務
```

第**3**個月：專案實戰

這時候，你應該要有3-5個小專案了。

選一個你最驕傲的，把它做到最好：

- 加上錯誤處理
- 寫好註解文件
- 放上GitHub
- 寫一篇部落格分享

關鍵決策點

三個月後，你會面臨選擇。

路線A：深化現有技能

留在舒適圈邊緣，但持續進步。

「我要把Python學到精通。」

行動計畫：

- 深入學習一個框架
- 參加線上課程
- 解決更複雜的問題
- 保持每日練習

路線B：橫向擴展

學習相關技術，成為T型人才。

「我要學前端來配合我的後端技能。」

行動計畫：

- 學習互補技術
- 做全端專案
- 了解完整流程
- 建立技術廣度

路線C：準備轉職

全力以赴，改變職涯軌道。

「我要成為專業工程師。」

行動計畫：

- 每日學習4小時以上
- 建立完整作品集
- 刷LeetCode題目
- 準備面試

沒有對錯，只有適合。

避開陷阱

我看過太多人在這些地方跌倒：

陷阱**1**：教學地獄

```
症狀：看了100個教學，沒寫過一個專案
解方：看一個教學，做三個練習
```

陷阱**2**：完美主義

```
症狀：程式碼改了又改，專案永遠沒完成
解方：設定期限，時間到就發布
```

陷阱**3**：比較心態

```
症狀：看到別人的作品就沮喪
解方：只跟昨天的自己比
```

陷阱**4**：獨自奮戰

```
症狀：遇到問題就卡住，沒人討論
解方：加入社群，找學習夥伴
```

陷阱**5**：貪多嚼不爛

```
症狀：同時學5種語言，每種都半調子
解方：專注一個，學到能用
```

建立你的學習系統

成功的學習者都有系統。

每日儀式

```
def 每日學習():
    # 早上（5分鐘）
    複習昨天的程式碼()

    # 通勤（15分鐘）
    閱讀技術文章()

    # 午休（20分鐘）
    寫一個小功能()

    # 晚上（30分鐘）
    專案開發()

    # 睡前（10分鐘）
    寫學習日記()
```

週末專案日

每個週末，給自己2-4小時的完整時間。

不是學習，而是創造。

用這週學的東西，做一個小作品。

月度回顧

每個月最後一天，問自己：

- 這個月學了什麼？
- 寫了哪些程式？
- 解決了什麼問題？
- 下個月要挑戰什麼？

找到你的節奏

小美的節奏：早起學習 「早上5:30-6:30，家人還在睡，這是我的程式時間。」

阿威的節奏：深夜創作 「晚上11點後最有靈感，一寫就到凌晨2點。」

老陳的節奏：午休充電 「中午12:30-13:30，一半吃飯，一半寫程式。」

小李的節奏：通勤學習 「捷運上40分鐘，用手機看教學，回家直接練習。」

你的節奏是什麼？

不要複製別人的，找到適合你的。

第一年後

如果你堅持了一年，你會：

- 寫過至少10,000行程式碼
- 完成10-20個專案
- 解決無數個bug
- 認識一群同好
- 建立程式思維
- 可能找到新工作
- 一定找到新自信

但最重要的是，你會發現：

學程式不是終點，而是起點。

它打開了一扇門，門後是無限可能。

現在，立刻，馬上

別等明天。別等下週一。別等新年。

現在就開始。

打開電腦。 打開瀏覽器。 搜尋「Python download」或「Node.js download」。 安裝。 打開終端機。 輸入：

```
print("Hello, World!")
print("我的程式之旅，現在開始。")
```

執行。

看到了嗎？那兩行字？

那是你寫的第一個程式。

從這一刻起，你就是程式設計師了。

不是未來式，是現在式。

最後的話

記得第一章那個深夜的自己嗎？

那個盯著螢幕、不知道程式是什麼的你？

現在你知道了。

程式是思考的工具。 程式是創造的畫筆。 程式是自動化的魔法。 程式是解決問題的鑰匙。

但最重要的——

程式是你證明自己「可以學會任何事」的證據。

如果你能從完全不懂到寫出程式，那還有什麼是你做不到的？

寫給一年後的你

一年後，當你回頭看這本書，你可能會笑。

笑那個連變數都不懂的自己。 笑那個為了一個分號苦惱三小時的自己。 笑那個第一次成功印出"Hello World"就興奮的自己。

但請記得，那個什麼都不懂卻願意開始的自己，是最勇敢的。

記得今天的感覺。 記得第一次的興奮。 記得突破的喜悅。

因為學習永無止境，你會一直需要這份初心。

寫在全書的最後

故事要結束了，但你的故事才正要開始。

小美後來成了公司的數位轉型推手。她不只自己寫程式，還教其他同事。

阿威開了自己的互動設計工作室。他的作品既美觀又有強大的功能。

老陳在55歲那年，成功轉職成為金融科技顧問。

小李？他現在是一家新創公司的技術長。

他們都不是天才。 他們只是開始了，然後堅持下去。

就像你即將做的一樣。

這不是結束，這是開始。

歡迎來到程式的世界。 歡迎來到創造者的行列。 歡迎成為那個你想成為的人。

現在，關上這本書。 打開你的電腦。 開始寫你的第一行程式碼。

你的故事，從此刻開始。

加油。

我在程式的世界裡等你。

「*The future belongs to those who learn more skills and combine them in creative ways.*」 *— Robert Greene*

未來屬於那些不斷學習並創意組合技能的人。

而你，正是其中之一。

全書完

謝謝你的閱讀。 謝謝你的信任。 謝謝你願意開始。

現在，去創造屬於你的精彩吧。

感謝你讀到這裡

> 每一個程式大師，都曾經是完全不懂的小白。
>
> 你今天翻開這本書，就已經比昨天的自己更接近目標了。

這不是結束，而是開始

學習程式的路上，你不需要孤軍奮戰。

桑尼資料科學致力於用最白話的方式，幫助每個人踏入程式與數據的世界。

無論你是：

- 想轉職的上班族
- 想提升效率的行銷人
- 對程式好奇的學生
- 或是任何想改變的人

我們都歡迎你。

加入桑尼的學習社群

追蹤 **Instagram**

獲取最新的學習技巧、程式小知識、職涯分享

@datasunnie

加入 **LINE** 官方帳號

第一手課程資訊、免費學習資源、問題解答

@055zjitb

期待你的第一步

1. 立即追蹤 — 掃描上方 QR Code
2. 開始練習 — 用第10章的行動計畫開始你的第一步
3. 持續學習 — 我們會陪你一起成長

王子公主請上車，從這裡啟程

桑尼資料科學 **| Data Sunnie**

讓數據說故事，讓程式改變生活

> 特別感謝
>
> 感謝你選擇這本書作為程式學習的起點。
>
> 如果這本書對你有幫助，歡迎分享給身邊也想學程式的朋友。
>
> 我們下次見！

> — Sunnie